CRISP: Compressed Reasoning via Iterative Self-Policy Distillation

Hejian Sang[‡] hejian@alumni.iastate.edu
Yuanda Xu[*] yuanda@math.princeton.edu
Zhengze Zhou[*] zz433@cornell.edu
Ran He[*] rh2528@columbia.edu
Zhipeng Wang zhipeng.wang@alumni.rice.edu
Jiachen Sun jiachens@umich.edu

Abstract

Reasoning models think out loud, but much of what they say is noise. We introduce CRISP (**C**ompressed **R**easoning via **I**terative **S**elf-**P**olicy Distillation), a method that teaches models to reason more concisely by distilling their own concise behavior back into themselves. The entire approach reduces to one idea: condition the same model on a "be concise" instruction to obtain teacher logits, and minimize per-token reverse KL on the student's own rollouts. No ground-truth answers, no token budgets, no difficulty estimators. Just self-distillation. Yet this simplicity belies surprising sophistication: CRISP automatically compresses easy problems aggressively while preserving the deliberation needed for hard ones. On Qwen3-8B and Qwen3-14B, we achieve 57–59% token reduction on MATH-500 while *improving* accuracy by 9–16 points absolute. On AIME 2024, the 14B model gains 10 points with 41% compression. Ablations show that qualitative conciseness instructions outperform explicit token targets, and periodic teacher refreshes yield a broad stable regime. The method generalizes across model families—DeepSeek-R1-Distill-Llama-8B improves accuracy by up to 5 points with 17–32% compression—and transfers beyond math to multi-step agentic planning (DeepPlanning), reducing token usage by 42–51% while preserving planning quality.

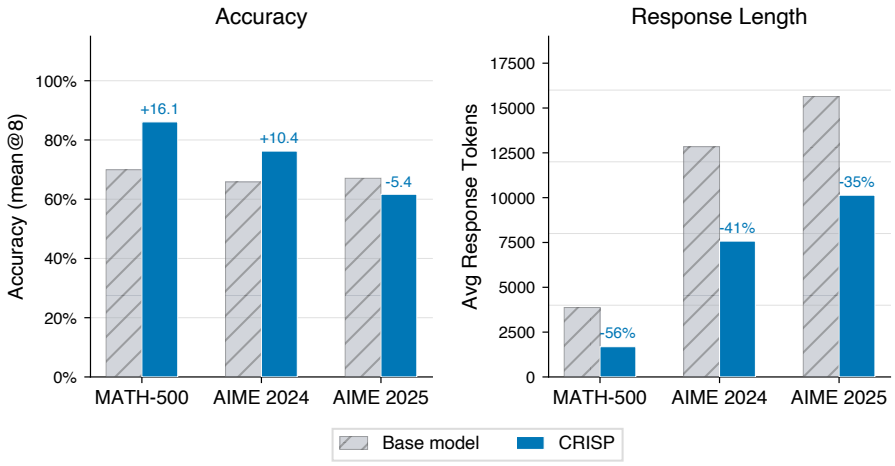

Figure 1: **The paradox of reasoning compression: less thinking, better answers.** Results for Qwen3-14B across three benchmarks of increasing difficulty (30K response token budget). CRISP compresses reasoning traces by 35–57% while largely preserving or *improving* accuracy, most dramatically on MATH-500, where accuracy jumps from 70.0% to 86.1%.

[*]Equal contribution.
[†]Correspondence to hejian@alumni.iastate.edu

1 Introduction

Modern reasoning models have learned to think before they speak, and they have a lot to say. Systems like OpenAI o1 (Jaech et al., 2024), Gemini 2.5 (Comanici et al., 2025), DeepSeek-R1 (Guo et al., 2025), and Qwen3 (Yang et al., 2025) produce thousands of tokens of internal deliberation before arriving at an answer, exploring blind alleys, second-guessing themselves, and verifying conclusions. This verbosity pays off on hard problems. But it comes at a cost: these models *cannot stop talking*, even when the answer is obvious. Ask them what $2 + 2$ is, and they may spend 500 tokens considering whether you meant binary arithmetic (Snell et al., 2024; Muennighoff et al., 2025). That being said, the computational overhead stem from the "overthinking" does not help on the quality of answers for many problems.

The community has noticed the limitations. A flurry of reasoning compression methods has emerged (Appendix D surveys some recent approaches), each attacking the problem from a different angle. But every existing paradigm demands a sacrifice: RL methods need ground-truth answers and risk collapsing the model's ability to explore (Aggarwal & Welleck, 2025; Wan et al., 2026; Liu et al., 2025); SFT methods train on someone else's reasoning and forget their own (Huang et al., 2025; Shenfeld et al., 2026); most treat all problems alike, compressing a trivial sum as aggressively as a competition integral; and prompting tricks vanish the moment you remove the prompt.

We propose CRISP (**C**ompressed **R**easoning via **I**terative **S**elf-**P**olicy Distillation), a method that sidesteps all of these trade-offs with a single, almost trivial idea: *ask the model to be concise, then teach it to do so without being asked*. The model already knows how to compress; it just needs permission. We give it that permission via a conciseness instruction, then distill this behavior back into the base model. No rewards, no budgets, no oracles. Given a reasoning model π_θ, we define:

- **Teacher**: $\pi_\theta(\cdot \mid x, c)$, the same model conditioned on a conciseness instruction c (for example: "Solve concisely, avoid unnecessary steps").
- **Student**: $\pi_\theta(\cdot \mid x)$, the same model without the compression instruction.

Training generates student rollouts and minimizes the per-token reverse KL divergence between student and teacher distributions. This on-policy self-distillation approach requires no ground-truth answers, no reward engineering, and no difficulty estimation. The compression signal emerges naturally from the KL objective, adapting automatically to problem difficulty.

Table 1: **Comparison of reasoning compression methods.** CRISP uniquely combines on-policy training, no dependence on ground-truth (GT) answers, difficulty-adaptive compression, and entropy preservation.

Method	On-policy	No GT needed	Difficulty-adaptive	Entropy-preserving
RL + length penalty (Aggarwal & Welleck, 2025; Wan et al., 2026)	✓	✗	✗	✗
SFT on compressed CoT (Huang et al., 2025)	✗	✗	✗	✓
OPCD (Ye et al., 2026)	✓	✗	✗	✓
DLER (Liu et al., 2025)	✓	✗	✗	✗
Prompting / pruning (Xu et al., 2025)	—	✓	✗	✓
CRISP (ours)	✓	✓	✓	✓

Table 1 contrasts CRISP with representative methods from each paradigm. CRISP is the only approach that satisfies all four desiderata.

Summary of results. On Qwen3-8B and Qwen3-14B, CRISP achieves **57–59% token reduction on MATH-500 while improving accuracy by 9–16 percentage points** (to ∼86%). On AIME 2024, the 14B model gains 10 points with 41% compression. Compression naturally

adapts to difficulty ($\sim$1.6$\times$ more compression on easy vs. hard problems), entropy remains stable throughout training, and general capabilities (MMLU) are fully preserved. Our ablations also show that the default recipe is principled rather than accidental: qualitative "be concise" instructions outperform explicit percentage targets, periodic teacher refreshes admit a broad stable regime at $M \in \{40, 50, 60\}$, and reverse KL is markedly more stable than forward KL for iterative on-policy self-distillation. Beyond math benchmarks, the method generalizes in two important dimensions. **Cross-model:** applying the same recipe to DeepSeek-R1-Distill-Llama-8B (a different model family) yields accuracy gains of up to 5 points with 17–32% token reduction, confirming that the approach is not specific to Qwen. **Cross-task:** the compressed Qwen3-14B model transfers to multi-step agentic planning on DeepPlanning, reducing response length by 42–51% while preserving planning quality.

2 Related Work

Reasoning compression via reinforcement learning. The most direct approach: penalize length in the reward function. L1 (Aggarwal & Welleck, 2025) caps token count during GRPO training. DiPO (Wan et al., 2026) and DIET (Chen et al., 2025b) estimate difficulty from rollout pass rates and set per-problem length targets. Leash (Li et al., 2025b) shapes rewards with sigmoid functions; DLER (Liu et al., 2025) adds curriculum learning. ThinkPrune (Hou et al., 2025) continuously trains long-thinking LLMs using reinforcement learning (RL) with an additional token-budget constraint while preserving answer correctness. The catch: all of these require ground-truth answers. No correct answer, no reward and no way to know if compression went too far. Xu et al. (2026a) further notes that reinforcement learning can induce overconfidence errors, thereby narrowing the model's reasoning boundary and reducing generation diversity.

Reasoning compression via supervised fine-tuning. Another route: curate short reasoning traces, then train on them. SEER (Huang et al., 2025) samples many solutions and keeps the shortest correct ones. TokenSkip (Xia et al., 2025) learns which tokens to skip. DAP/LiteCoT (Wu et al., 2025) distills from stronger models; S3-CoT (Du et al., 2026) steers activations toward brevity. The problem is distribution shift: the student trains on someone else's reasoning and forgets its own (Shenfeld et al., 2026).

Training-free compression. The lightweight option: change the prompt or the decoder, not the weights. Chain of Draft (Xu et al., 2025) asks for minimal drafts instead of full reasoning. TrimR (Lin et al., 2025) prunes after the fact. NoWait (Wang et al., 2025a) and FlowSteer (Li et al., 2026) steer decoding toward conciseness. These methods are easy to deploy but achieve limited compression, and the effect vanishes when you change the prompt.

On-policy self-distillation. The closest relatives of our work use the model as its own teacher. OPSD (Zhao et al., 2026) gives the teacher the ground-truth answer, achieving 4–8$\times$ efficiency over GRPO. SDPO (Hübotter et al., 2026) conditions on rich feedback for dense credit assignment. SDFT (Shenfeld et al., 2026) shows that on-policy distillation dramatically reduces forgetting compared to standard SFT, interpreting it as inverse RL. PACED (Xu et al., 2026b) studies competence-aware self-distillation via pass-rate weighting. OPCD (Ye et al., 2026) distills system-prompt behaviors into weights. We contribute a new application: using a *conciseness instruction* as the privileged context, achieving compression without any ground-truth supervision.

3 Method

3.1 Problem Formulation

Consider a reasoning model π_θ that, given input x, generates a reasoning trace r followed by an answer a, producing output $y = (r, a)$. The reasoning trace typically appears within

<think>...</think> delimiters. We aim to learn parameters θ^* such that the model produces shorter reasoning traces while maintaining accuracy.

Let c denote a conciseness instruction. The student receives the original DAPO-17K math prompt, while the teacher receives the same prompt prefixed with a conciseness instruction. We denote them by $\pi_\theta(\cdot \mid x)$ and $\pi_\theta(\cdot \mid x, c)$; full templates are provided in Appendix B.

3.2 Training Objective

CRISP minimizes the per-token reverse KL divergence between the student and a stop-gradient teacher on student-generated rollouts:

$$\mathcal{L}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)} \left[\sum_{t=1}^{|y|} D_{\mathrm{KL}}\Big(\pi_\theta(\cdot \mid x, y_{<t}) \;\Big\|\; \pi_{\bar{\theta}}(\cdot \mid x, c, y_{<t}) \Big) \right], \tag{1}$$

where $\bar{\theta}$ denotes the teacher weights, which are periodically synchronized with the student (Section 3.3), and no gradients flow through the teacher's forward pass. The expectation over $y \sim \pi_\theta(\cdot \mid x)$ makes training *on-policy*: the student is optimized on its own generation distribution, which prevents the distribution shift inherent in off-policy SFT.

Why reverse KL? The divergence direction matters because teacher refreshes make training iterative rather than one-shot. Reverse KL ($D_{\mathrm{KL}}(\pi_\theta \| \pi_{\bar{\theta}})$) weights updates by the student's own distribution, so the model only changes in regions it already visits, which stabilizes periodic teacher refreshes. Forward KL instead weights updates by the teacher distribution and empirically overreacts to refreshes, causing aggressive over-compression and accuracy collapse on hard benchmarks (Appendix I).

3.3 Teacher Parameterization

A natural baseline is a *fully frozen* teacher ($\bar{\theta} = \theta_0$) as in Zhao et al. (2026). While simple and stable, the frozen teacher becomes an increasingly weak compression target as the student improves: once the student has internalized the initial conciseness signal, no further compression is possible because the reference distribution no longer leads the student.

To address this, we adopt a *periodic teacher update* strategy. The teacher weights are synchronized with the current student weights every M training steps:

$$\bar{\theta} \leftarrow \theta \quad \text{every } M \text{ steps.} \tag{2}$$

Each refresh creates a new, stronger compression target: the updated teacher, when conditioned on the conciseness instruction c, produces traces that are more concise than the previous teacher's (since the student, now serving as the new teacher, has already learned to compress). This *progressive compression* effect pushes the student to continuously shorten its reasoning over the course of training, beyond what a single frozen reference can achieve.

Difficulty-adaptive compression. Compression adapts naturally to problem difficulty: for easy problems, the concise teacher produces much shorter traces, creating strong KL signal; for hard problems, even the teacher needs extensive reasoning, yielding weak signal. We formalize this in Proposition 2 and verify it empirically in Section 5.3.

3.4 Training Algorithm

The complete CRISP training procedure is given in Algorithm 1.

Computational cost and simplicity. The pipeline requires only standard supervised training infrastructure—no reward models, no value functions, no multi-rollout sampling. Each step requires two forward passes per rollout token (student with gradient, teacher without), and the periodic refresh is a simple weight copy. This yields substantial efficiency gains over RL methods, which require multiple rollouts per prompt, reward model inference, and complex optimization (e.g., PPO clipping, GAE).

Algorithm 1: CRISP: On-Policy Self-Distillation for Concise Reasoning

Input: Model π_θ, dataset $\mathcal{D} = \{x_i\}$, conciseness instruction c, learning rate η, teacher update interval M
Output: Compressed reasoning model π_{θ^*}
Initialize teacher: $\bar{\theta} \leftarrow \theta_0$;
for *each training step* $k = 1, 2, \ldots$ **do**
  if $k \bmod M = 0$ **then**
    Update teacher: $\bar{\theta} \leftarrow \theta$; `// periodic refresh`
  end
  Sample batch $\{x_1, \ldots, x_B\} \sim \mathcal{D}$;
  for *each* x_i *in batch* **do**
    Generate student rollout: $y_i \sim \pi_\theta(\cdot \mid x_i)$;
    for *each token position* $t = 1, \ldots, |y_i|$ **do**
      Compute student logits: $q_t \leftarrow \pi_\theta(\cdot \mid x_i, y_{i,<t})$;
      Compute teacher logits: $p_t \leftarrow \pi_{\bar{\theta}}(\cdot \mid x_i, c, y_{i,<t})$; `// no grad`
      Compute $D_{\mathrm{KL}}(q_t \| p_t)$;
    end
    $\mathcal{L}_i \leftarrow \sum_t D_{\mathrm{KL}}(q_t \| p_t)$;
  end
  Update student: $\theta \leftarrow \theta - \eta \nabla_\theta \frac{1}{B} \sum_i \mathcal{L}_i$;
  ; `// normalized by` $|y_i|$ `in practice`
end
return π_{θ^*};

4 An Interpretive Framework

The primary contributions of this paper are methodological and empirical. In this section we complement them with an interpretive framework that makes the mechanisms behind CRISP more transparent. We keep only the main takeaways here and defer all definitions and proofs to Appendix A. The framework serves four purposes: it identifies the training loss as a sequence-level reverse KL, interprets this KL as an implicit conciseness reward, explains why compression can preserve accuracy and avoid forgetting, and clarifies why shorter traces can sometimes improve accuracy. None of these results require novel proof techniques; their value lies in connecting CRISP's design choices to well-understood information-theoretic quantities and in guiding practitioner intuition.

By the autoregressive chain rule, the per-token objective in Eq. 1 is exactly the sequence-level divergence $D_{\mathrm{KL}}(\pi_\theta(\cdot \mid x) \| \pi_{\bar{\theta}}(\cdot \mid x, c))$ (Lemma 1). This lets us analyze CRISP as matching the student's full rollout distribution to that of the concise teacher on the student's own trajectories.

Implicit reward. The CRISP objective (Eq. 1) is equivalent to maximizing the expected implicit reward

$$r(y_t, x) = \log \pi_{\bar{\theta}}(y_t \mid x, c, y_{<t}) - \log \pi_\theta(y_t \mid x, y_{<t}). \tag{3}$$

This reward is positive on tokens the concise teacher prefers and negative on tokens the student overproduces. In effect, CRISP suppresses unnecessary reasoning without introducing an explicit length penalty or a correctness verifier.

Accuracy preservation (Proposition 1). If training converges to loss ϵ_{KL} and the concise teacher preserves accuracy to within ϵ_T of the base model, the student satisfies

$$\mathrm{Acc}(\pi_{\theta^*}) \geq \mathrm{Acc}(\pi_{\bar{\theta}}) - \epsilon_T - \sqrt{\epsilon_{\mathrm{KL}}/2}. \tag{4}$$

The bound cleanly separates teacher quality from distillation gap. In our setting, the concise teacher is often more accurate than the base model, so compression can improve accuracy rather than merely preserve it.

Difficulty-adaptive compression (Proposition 2). The compression signal is non-increasing in problem difficulty: easy problems receive stronger pressure to shorten reasoning, while hard problems receive weaker pressure because a larger fraction of their tokens are essential. This formalizes the empirical pattern that CRISP compresses MATH-500 much more aggressively than AIME without any explicit difficulty estimator.

Bounded forgetting (Proposition 3). Divergence from the base model is bounded by

$$\mathbb{E}_x\left[d_{\mathrm{TV}}(\pi_{\theta^*}(\cdot \mid x),\, \pi_{\theta_0}(\cdot \mid x))\right] \le \sqrt{\epsilon_{\mathrm{KL}}/2} + \mathbb{E}_x[\gamma(x)], \tag{5}$$

where $\gamma(x)$ is the conciseness gap, i.e., the total variation between the base model's outputs with and without the conciseness instruction. For hard problems, the instruction changes the base model very little, so $\gamma(x)$ is small and forgetting is correspondingly limited.

Compression reduces compounding error (Proposition 4). Under a model where each token independently introduces a reasoning error with probability p_{err}, compressing from L to αL tokens yields an accuracy ratio $(1 - p_{\mathrm{err}})^{-(1-\alpha)L}$, which grows exponentially in the number of removed tokens. This gives a simple explanation for the empirical "less is more" effect: when long chains contain redundant or fragile steps, removing tokens can reduce the chance that the model talks itself into an error.

5 Experiments

5.1 Experimental Setting

Models and data. We evaluate CRISP on Qwen3-8B and Qwen3-14B (Yang et al., 2025), training on ∼13,600 competition-level math problems from DAPO-Math-17k (Yu et al., 2025) *without ground-truth answers*; only problem statements are used to generate student rollouts. We train for 1 epoch with learning rate 1×10^{-6}, global batch size 32, periodic teacher update (interval M=50; see ablation in Section C.2), and 8× H200 GPUs. Although nominally a full epoch, the algorithm converges quickly at around ∼100 steps. Each prompt generates a single student rollout (temperature 1.0) with a maximum response length of 8,192 tokens. Because CRISP optimizes a per-token KL objective rather than an outcome-based reward, partial rollouts already provide a useful training signal, unlike RL methods that require complete responses (Chen et al., 2025a). Full training and infrastructure details are in Appendix F.

Benchmarks. We evaluate on three mathematical reasoning benchmarks spanning a wide difficulty range: MATH-500 (Hendrycks et al., 2021) (500 problems, base accuracy 70–78%), AIME 2024 (30 problems, 66–73%), and AIME 2025 (30 problems, 63–67%).[1] We define a *token budget* as the maximum response length allowed during inference, a practical lever for controlling serving cost. We report results under two budgets: 8,192 tokens, representative of efficient serving constraints, and 30,000 tokens, which effectively eliminates truncation and enables fairer accuracy comparison.

5.2 Self-Distillation Simultaneously Compresses and Improves Reasoning

Table 2 presents our main results under the 30,000-token budget, which eliminates response truncation for a fair accuracy comparison.[2]

[1] All benchmarks are evaluated using the math answer grading utility from veRL (Sheng et al., 2025): https://github.com/verl-project/verl/blob/main/verl/utils/reward_score/math_dapo.py.

[2] MMLU is evaluated using the Language Model Evaluation Harness (Gao et al., 2021): https://github.com/EleutherAI/lm-evaluation-harness.

Table 2: **Self-distillation compresses reasoning traces while improving accuracy without forgetting (token budget = 30K).** Results on Qwen3-8B and Qwen3-14B with a 30,000-token budget to eliminate truncation effects. Accuracy (Acc, mean over 8 samples per problem, %), average reasoning token length (Len), and token reduction relative to the base model (Red., %). "Concise prompt" uses the conciseness instruction at inference only (no training); CRISP trains with periodic teacher update (M=50). The rightmost column reports MMLU (Hendrycks et al., 2020) accuracy to verify that general capabilities are preserved. Results under the efficient-serving budget (8,192 tokens) are in Table 6 (Appendix E.1).

	MATH-500			AIME 2024			AIME 2025			MMLU
Method	Acc	Len	Red.	Acc	Len	Red.	Acc	Len	Red.	Acc
Qwen3-8B										
Base Model	77.7	4,661	—	**72.5**	14,170	—	**62.5**	16,682	—	73.2
Concise prompt	80.9	2,941	36.9%	67.5	11,589	18.2%	56.3	14,347	14.0%	—
CRISP	**86.6**	1,921	**58.8%**	69.6	9,152	**35.4%**	57.1	10,726	**35.7%**	73.3
Qwen3-14B										
Base Model	70.0	3,872	—	65.8	12,844	—	**67.1**	15,642	—	76.9
Concise prompt	83.8	2,426	37.3%	68.3	9,866	23.2%	58.3	12,831	18.0%	—
CRISP	**86.1**	1,686	**56.5%**	**76.3**	7,577	**41.0%**	61.7	10,137	**35.2%**	76.9

Finding 1: Less is more. CRISP simultaneously improves accuracy *and* reduces reasoning length on MATH-500, the opposite of the trade-off everyone assumes. Both models reach ∼86% accuracy (from 77.7% for 8B and 70.0% for 14B) while shedding **57–59%** of their tokens. On AIME 2024, the 14B model improves substantially (65.8→76.3, +10.5pp) with 41% compression. On the harder AIME 2025, accuracy decreases modestly (∼5pp) as the model trades some deliberation for efficiency.

The concise prompt alone (no training) already improves MATH-500 accuracy while reducing tokens by 14–37%, confirming that the base model's reasoning contains substantial redundancy; CRISP amplifies both effects. Crucially, MMLU accuracy is fully preserved (73.2→73.3 for 8B, 76.9→76.9 for 14B), confirming that self-distillation does not degrade general capabilities.

5.3 Compression Naturally Adapts to Problem Difficulty

Finding 2: Compression naturally adapts to difficulty. On the easiest benchmark (MATH-500), CRISP compresses by **56.5–58.8%**. On the hardest (AIME 2025), compression drops to **35.2–35.7%**, a ∼1.6× **ratio** that emerges automatically from the KL objective (see §3.3).

Larger models gain more from distillation. Qwen3-14B starts from *lower* base accuracy on MATH-500 (70.0% vs. 77.7% for 8B) yet achieves comparable post-distillation accuracy (86.1% vs. 86.6%). On AIME 2024, the 14B model *improves* by 10.4 points while the 8B model slightly declines. Larger models follow instructions better, making the concise teacher a stronger signal, and they have more redundancy to shed.

5.4 Self-Distillation Does Not Collapse Model Entropy

Finding 3: Self-distillation preserves what RL destroys. A central concern with reasoning compression is *entropy collapse*: RL methods with length penalties systematically suppress high-entropy "exploratory" tokens ("Wait," "Alternatively," "Let me reconsider...") that are critical for solving hard problems (Wang et al., 2025b). Figure 8 (Appendix H.1) shows that CRISP avoids this entirely—entropy remains stable throughout training. The model learns to *choose* conciseness rather than being forced into it.

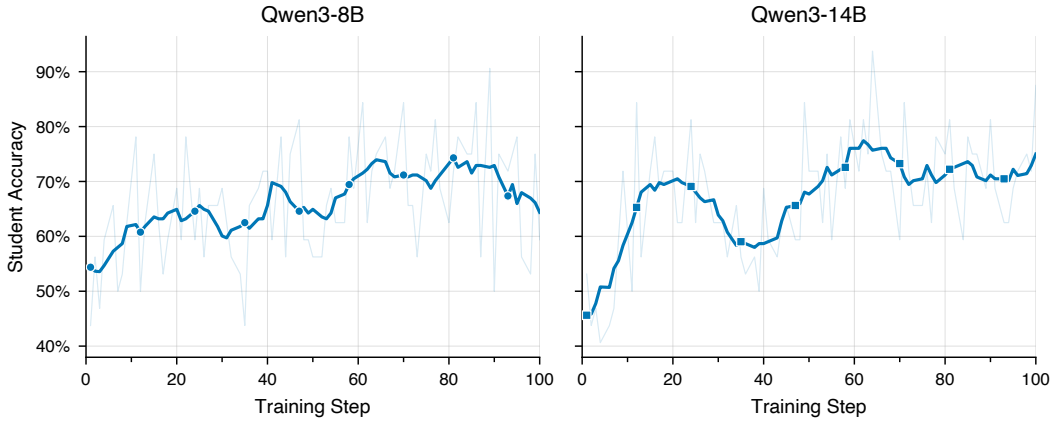

Figure 2: **Student mean accuracy on training data increases during self-distillation.** Qwen3-8B improves from ∼52% to ∼66% and Qwen3-14B from ∼46% to ∼72%, despite no correctness reward. The concise teacher's implicit reward reshapes the student's output distribution, concentrating probability mass on direct, correct reasoning paths.

This follows from mode-seeking reverse KL (§3.2): the student is penalized for placing mass where the teacher assigns low probability, but *not* for maintaining mass where the teacher is also uncertain. RL length penalties, by contrast, reward shorter outputs regardless of token informativeness.

5.5 Why Does Compression Improve Accuracy?

Finding 4: Compression improves accuracy via implicit reward shaping. The reverse KL objective implicitly rewards concise, correct tokens (Eq. 3) and concentrates probability mass on the teacher's preferred mode. Training-time accuracy increases monotonically (Figure 2)—from ∼52% to ∼66% for Qwen3-8B and ∼46% to ∼72% for Qwen3-14B—despite receiving no correctness reward.

The mode-seeking property of reverse KL (§3.2) drives the student toward the teacher's preferred mode (Gu et al., 2023; Li et al., 2025a), concentrating probability mass on direct, correct reasoning paths. Since each additional token is a potential point of failure (Chen et al., 2024)—an effect we formalize in Proposition 4—compression simultaneously shortens traces and reduces error accumulation. Figure 12 in Appendix K provides side-by-side comparisons.

Appendix C shows that qualitative "be concise" instructions outperform explicit percentage targets (which compress more but sacrifice accuracy), and that the teacher update interval admits a broad stable regime at $M \in \{40, 50, 60\}$, while overly frequent updates (M=1) cause collapse. Appendix I further shows that replacing reverse KL with forward KL introduces progressively deeper regressions after each teacher refresh.

5.6 CRISP Generalizes Across Model Families

To evaluate generalization beyond the Qwen model family, we apply CRISP to DeepSeek-R1-Distill-Llama-8B, a reasoning model distilled from DeepSeek-R1 (Guo et al., 2025) into the Llama-3.1-8B architecture. We use the same training recipe as in Section 5.1.

Table 3: **CRISP on DeepSeek-R1-Distill-Llama-8B compresses reasoning traces while improving accuracy.** Accuracy (Acc, mean over 8 samples per problem, %), average reasoning token length (Len), and token reduction relative to the base model (Red., %). CRISP trains with periodic teacher update (M=50).

	MATH-500			AIME 2024			AIME 2025		
Method	Acc	Len	Red.	Acc	Len	Red.	Acc	Len	Red.
Distill-Llama-8B									
Base Model	61.3	3,102	—	37.9	11,259	—	27.5	11,214	—
CRISP	**62.7**	2,104	**32.2%**	**42.9**	9,311	**17.3%**	**32.1**	9,939	**11.4%**

Finding 5: CRISP generalizes across model families and task domains. As shown in Table 3, applying the identical recipe to DeepSeek-R1-Distill-Llama-8B improves accuracy on all three math benchmarks (+1.4 on MATH-500, +5.0 on AIME 2024, +4.6 on AIME 2025) while reducing tokens by 11–32%. On the non-math DeepPlanning benchmark, compressed Qwen3-14B models reduce agentic response tokens by 42–51% while preserving travel-planning and shopping-planning quality (Figure 11 in Appendix J). These results confirm that CRISP is not specific to one model family or to mathematical reasoning.

Why is compression smaller than on Qwen models? The 32% token reduction on MATH-500 is notably lower than the 57–59% achieved by Qwen3-8B/14B. We attribute this to two factors. First, Distill-Llama-8B already produces substantially shorter base responses (3,102 tokens vs. 4,661 for Qwen3-8B on MATH-500), leaving less redundancy to remove. Second, the Llama architecture appears less responsive to the qualitative conciseness instruction: because DeepSeek-R1-Distill-Llama-8B was trained on DeepSeek-R1 traces rather than instruction-following data, the gap between its base and concise-prompted distributions is smaller, yielding a weaker distillation signal. Despite these differences, the consistent accuracy gains across all three benchmarks confirm that CRISP transfers across model families, with compression magnitude naturally adapting to the amount of available redundancy.

6 Limitations and Future Work

Instruction-following as an enabler. CRISP requires reasonably strong instruction-following ability; smaller models may fail to respond reliably to a conciseness instruction, which likely explains why larger models benefit more. Characterizing the minimum capability threshold is an important open question.

Teacher quality characterization. Our experiments show that the conciseness-conditioned teacher improves accuracy (Table 2), and Proposition 1 provides formal bounds. A finer-grained characterization of when and why conciseness instructions improve versus degrade accuracy across model families would further strengthen understanding of self-distillation dynamics.

7 Conclusion

CRISP shows that much of what reasoning models produce is not deliberation but *noise*—and noise compounds. Every unnecessary token is a chance to wander off course, to second-guess a correct answer, to introduce an error that propagates forward. By teaching models to skip the noise, we do not sacrifice depth; we *recover* it.

Two takeaways stand out. First, verbosity is not caution—it can be a source of compounding error. Second, models already possess a latent ability to be concise; on-policy self-distillation can make this behavior the default without sacrificing entropy or general capabilities.

Finally, CRISP's supervision is purely behavioral: a conciseness instruction and the model's own rollouts. This suggests a path to compressing reasoning in domains where ground-truth answers or reliable verifiers are unavailable, as long as the model can follow the desired instruction.

References

Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.

Wei-Rui Chen, Vignesh Kothapalli, Ata Fatahibaarzi, Hejian Sang, Shao Tang, Qingquan Song, Zhipeng Wang, and Muhammad Abdul-Mageed. Distilling the essence: Efficient reasoning distillation via sequence truncation. *arXiv preprint arXiv:2512.21002*, 2025a.

Weize Chen, Jiarui Yuan, Tailin Jin, Ning Ding, Huimin Chen, Zhiyuan Liu, and Maosong Sun. The overthinker's diet: Cutting token calories with difficulty-aware training. *arXiv preprint arXiv:2505.19217*, 2025b.

Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, et al. Do not think that much for 2+ 3=? on the overthinking of o1-like llms. *arXiv preprint arXiv:2412.21187*, 2024.

Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.

Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.

Yanrui Du, Sendong Zhao, Yibo Gao, Danyang Zhao, Qika Lin, Ming Ma, Jiayun Li, Yi Jiang, Kai He, Qianyi Xu, et al. S3-cot: Self-sampled succinct reasoning enables efficient chain-of-thought llms. *arXiv preprint arXiv:2602.01982*, 2026.

Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, et al. A framework for few-shot language model evaluation, 2021.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *arXiv preprint arXiv:2306.08543*, 2023.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning. *arXiv preprint arXiv:2504.01296*, 2025.

Kerui Huang, Shuhan Liu, Xing Hu, Tongtong Xu, Lingfeng Bao, and Xin Xia. Reasoning efficiently through adaptive chain-of-thought compression: A self-optimizing framework. *arXiv preprint arXiv:2509.14093*, 2025.

Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

Long Li, Jiaran Hao, Jason Klein Liu, Zhijian Zhou, Yanting Miao, Wei Pang, Xiaoyu Tan, Wei Chu, Zhe Wang, Shirui Pan, et al. The choice of divergence: A neglected key to mitigating diversity collapse in reinforcement learning with verifiable reward. *arXiv preprint arXiv:2509.07430*, 2025a.

Yanhao Li, Lu Ma, Jiaran Zhang, Lexiang Tang, Wentao Zhang, and Guibo Luo. Leash: Adaptive length penalty and reward shaping for efficient large reasoning model. *arXiv preprint arXiv:2512.21540*, 2025b.

Yawei Li, Benjamin Bergner, Yinghan Zhao, Vihang Prakash Patil, Bei Chen, and Cheng Wang. Steering large reasoning models towards concise reasoning via flow matching. *arXiv preprint arXiv:2602.05539*, 2026.

Weizhe Lin, Xing Li, Zhiyuan Yang, Xiaojin Fu, Hui-Ling Zhen, Yaoyuan Wang, Xianzhi Yu, Wulong Liu, Xiaosong Li, and Mingxuan Yuan. Trimr: Verifier-based training-free thinking compression for efficient test-time scaling. *arXiv preprint arXiv:2505.17155*, 2025.

SY Liu, X Dong, X Lu, S Diao, M Liu, MH Chen, H Yin, YCF Wang, KT Cheng, Y Choi, et al. DLER: Doing length penalty right – incentivizing more intelligence per token via reinforcement learning. *arXiv preprint arXiv:2510.15110*, 2025.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori B Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 20286–20332, 2025.

Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Qian Wan, Ziao Xu, Luona Wei, Xiaoxuan Shen, and Jianwen Sun. Mitigating overthinking in large reasoning models via difficulty-aware reinforcement learning. *arXiv preprint arXiv:2601.21418*, 2026.

Chenlong Wang, Yuanning Feng, Dongping Chen, Zhaoyang Chu, Ranjay Krishna, and Tianyi Zhou. Wait, we don't need to" wait"! removing thinking tokens improves reasoning efficiency. *arXiv preprint arXiv:2506.08343*, 2025a.

Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. *arXiv preprint arXiv:2506.01939*, 2025b.

Yifan Wu, Jingze Shi, Bingheng Wu, Jiayi Zhang, Xiaotian Lin, Nan Tang, and Yuyu Luo. Concise reasoning, big gains: Pruning long reasoning trace with difficulty-aware prompting. *arXiv preprint arXiv:2505.19716*, 2025.

Heming Xia, Chak Tou Leong, Wenjie Wang, Yongqi Li, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 3351–3363, 2025.

Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025.

Yuanda Xu, Hejian Sang, Zhengze Zhou, Ran He, and Zhipeng Wang. Overconfident errors need stronger correction: Asymmetric confidence penalties for reinforcement learning. *arXiv preprint arXiv:2602.21420*, 2026a.

Yuanda Xu, Hejian Sang, Zhengze Zhou, Ran He, and Zhipeng Wang. Paced: Distillation and self-distillation at the frontier of student competence, 2026b. URL https://arxiv.org/abs/2603.11178.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. Deepplanning: Benchmarking long-horizon agentic planning with verifiable constraints. *arXiv preprint arXiv:2601.18137*, 2026.

Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

A Theoretical Analysis

We provide a formal analysis of CRISP's key properties: accuracy preservation guarantees (Section A.3), formalization of difficulty-adaptive compression (Section A.4), forgetting bounds relative to the base model (Section A.5), and a probabilistic model of how compression reduces compounding errors (Section A.6). We begin with two standard preliminary results that underpin the later proofs (Section A.1–A.2).

A.1 Preliminary: Sequence-Level Divergence

By the standard chain rule of KL divergence for autoregressive models, the per-token CRISP objective (Eq. 1) equals the sequence-level KL divergence between student and teacher.

Lemma 1 (Chain rule of KL for autoregressive models)**.** *For autoregressive distributions $q(y \mid x) = \prod_t q(y_t \mid x, y_{<t})$ and $p(y \mid x) = \prod_t p(y_t \mid x, y_{<t})$, the sequence-level KL decomposes as $D_{\mathrm{KL}}(q\|p) = \mathbb{E}_{y\sim q}[\sum_t D_{\mathrm{KL}}(q(\cdot \mid x, y_{<t})\|p(\cdot \mid x, y_{<t}))]$. This follows directly from expanding $\log q(y)/p(y) = \sum_t \log q(y_t \mid \cdot)/p(y_t \mid \cdot)$ and applying the tower property.*

Corollary 1. *Identifying $q = \pi_\theta(\cdot \mid x)$ and $p = \pi_{\tilde{\theta}}(\cdot \mid x, c)$, the CRISP training loss equals the expected sequence-level KL: $\mathcal{L}(\theta) = \mathbb{E}_{x\sim\mathcal{D}}[D_{\mathrm{KL}}(\pi_\theta(\cdot \mid x)\|\pi_{\tilde{\theta}}(\cdot \mid x, c))]$.*

A.2 Preliminary: Implicit Reward Interpretation

Following Shenfeld et al. (2026), the reverse-KL objective admits an implicit reward interpretation.

Proposition [Implicit reward, restated from §4]. The CRISP objective (Eq. 1) is equivalent to maximizing the expected implicit reward $r(y_t, x) = \log \pi_{\bar{\theta}}(y_t \mid x, c, y_{<t}) - \log \pi_\theta(y_t \mid x, y_{<t})$.

Proof of the implicit reward identity. Rewriting $\log(\pi_\theta / \pi_{\bar{\theta}}) = -r$ shows that the per-token reverse KL equals $-\mathbb{E}_{y_t \sim \pi_\theta}[r(y_t, x)]$. Summing over positions and applying Lemma 1 gives $\mathcal{L}(\theta) = -\mathbb{E}_{x, y \sim \pi_\theta}[\sum_t r(y_t, x)]$. □

The reward is positive on tokens the concise teacher prefers and negative on tokens the student overproduces, so CRISP suppresses verbosity without an explicit length penalty.

A.3 Accuracy Preservation under Compression

We show that if self-distillation converges (the training loss is small) and the concise teacher preserves accuracy, then the student's accuracy is guaranteed to remain close to the base model's.

Definition 1 (Accuracy)**.** For a problem distribution $\mathcal{D}$ with correct-answer sets $\{A(x)\}_{x \in \mathcal{D}}$, the accuracy of policy π is $\mathrm{Acc}(\pi) = \mathbb{E}_{x \sim \mathcal{D}}[\pi(A(x) \mid x)]$, where $\pi(A(x) \mid x) = \sum_{y \in A(x)} \pi(y \mid x)$.

Proposition 1 (Accuracy preservation)**.** *Let π_{θ^*} denote the converged student with training loss $\mathcal{L}(\theta^*) \le \epsilon_{\mathrm{KL}}$. Suppose the concise teacher preserves accuracy relative to the base model:*

$$\mathrm{Acc}(\pi_{\bar{\theta}}(\cdot \mid \cdot, c)) \ge \mathrm{Acc}(\pi_{\bar{\theta}}) - \epsilon_T. \tag{6}$$

Then the student satisfies:

$$\mathrm{Acc}(\pi_{\theta^*}) \ge \mathrm{Acc}(\pi_{\bar{\theta}}) - \epsilon_T - \sqrt{\frac{\epsilon_{\mathrm{KL}}}{2}}. \tag{7}$$

Proof. By Corollary 1, we have $\mathbb{E}_{x \sim \mathcal{D}}[D_{\mathrm{KL}}(\pi_{\theta^*}(\cdot \mid x) \| \pi_{\bar{\theta}}(\cdot \mid x, c))] \le \epsilon_{\mathrm{KL}}$.

Step 1: KL to total variation. For each problem x, Pinsker's inequality gives:

$$d_{\mathrm{TV}}(\pi_{\theta^*}(\cdot \mid x),\ \pi_{\bar{\theta}}(\cdot \mid x, c)) \le \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(\pi_{\theta^*}(\cdot \mid x) \| \pi_{\bar{\theta}}(\cdot \mid x, c))}. \tag{8}$$

Taking expectations over $x \sim \mathcal{D}$ and applying Jensen's inequality (using concavity of $\sqrt{\cdot}$):

$$\mathbb{E}_x[d_{\mathrm{TV}}(\pi_{\theta^*}(\cdot \mid x),\ \pi_{\bar{\theta}}(\cdot \mid x, c))] \le \sqrt{\tfrac{1}{2} \mathbb{E}_x[D_{\mathrm{KL}}(\pi_{\theta^*}(\cdot \mid x) \| \pi_{\bar{\theta}}(\cdot \mid x, c))]} \le \sqrt{\tfrac{\epsilon_{\mathrm{KL}}}{2}}. \tag{9}$$

Step 2: Total variation to accuracy. Since total variation bounds the difference in probability of any event, in particular the correctness event $A(x)$:

$$|\pi_{\theta^*}(A(x) \mid x) - \pi_{\bar{\theta}}(A(x) \mid x, c)| \le d_{\mathrm{TV}}(\pi_{\theta^*}(\cdot \mid x),\ \pi_{\bar{\theta}}(\cdot \mid x, c)). \tag{10}$$

Taking expectations over $x \sim \mathcal{D}$ and using $|\mathbb{E}[f]| \le \mathbb{E}[|f|]$:

$$|\mathrm{Acc}(\pi_{\theta^*}) - \mathrm{Acc}(\pi_{\bar{\theta}}(\cdot \mid \cdot, c))| \le \mathbb{E}_x[d_{\mathrm{TV}}(\pi_{\theta^*}(\cdot \mid x),\ \pi_{\bar{\theta}}(\cdot \mid x, c))] \le \sqrt{\tfrac{\epsilon_{\mathrm{KL}}}{2}}. \tag{11}$$

Step 3: Combine with teacher quality. From the assumption $\mathrm{Acc}(\pi_{\bar{\theta}}(\cdot \mid \cdot, c)) \ge \mathrm{Acc}(\pi_{\bar{\theta}}) - \epsilon_T$:

$$\mathrm{Acc}(\pi_{\theta^*}) \ge \mathrm{Acc}(\pi_{\bar{\theta}}(\cdot \mid \cdot, c)) - \sqrt{\tfrac{\epsilon_{\mathrm{KL}}}{2}} \ge \mathrm{Acc}(\pi_{\bar{\theta}}) - \epsilon_T - \sqrt{\tfrac{\epsilon_{\mathrm{KL}}}{2}}. \qquad \square$$

Remark 1 (When the bound is vacuous, and why that is informative)**.** *Proposition 1 identifies two independent sources of potential accuracy loss: the teacher accuracy gap ϵ_T and the student–teacher divergence $\sqrt{\epsilon_{\mathrm{KL}}/2}$. In our experiments, ϵ_T is consistently* negative *(the concise teacher*

is more accurate than the base model), making the bound vacuous in the traditional sense. This is informative rather than a weakness: it reveals why CRISP improves accuracy. The bound becomes $\mathrm{Acc}(\pi_{\theta^*}) \geq \mathrm{Acc}(\pi_{\bar{\theta}}) + |\epsilon_T| - \sqrt{\epsilon_{\mathrm{KL}}/2}$*, so accuracy improves whenever the teacher's accuracy gain exceeds the distillation gap. The theorem is most useful in the regime of aggressive compression where* ϵ_T *might turn positive; it then quantifies the worst-case accuracy degradation.*

A.4 Difficulty-Adaptive Compression

We formalize the empirical observation (Table 2) that CRISP compresses easy problems aggressively while preserving reasoning on hard problems.

Definition 2 (Essential and compressible tokens)**.** For problem x and student rollout $y \sim \pi_\theta(\cdot \mid x)$, classify each token position t based on the implicit reward sign (Eq. 3):

$$\mathcal{E}(x,y) = \{t : \pi_{\bar{\theta}}(y_t \mid x, c, y_{<t}) \geq \pi_\theta(y_t \mid x, y_{<t})\} \quad \text{(essential: } r(y_t, x) \geq 0\text{)}, \tag{12}$$

$$\mathcal{C}(x,y) = \{t : \pi_{\bar{\theta}}(y_t \mid x, c, y_{<t}) < \pi_\theta(y_t \mid x, y_{<t})\} \quad \text{(compressible: } r(y_t, x) < 0\text{)}. \tag{13}$$

Proposition 2 (Difficulty-adaptive compression signal)**.** *Let* $d(x) \in [0,1]$ *denote problem difficulty, defined as the base model's failure rate* $d(x) = 1 - \pi_{\theta_0}(A(x) \mid x)$*. Assume:*

(A1) ***Essential fraction increases with difficulty:*** *The expected fraction of essential tokens* $\rho(x) := \mathbb{E}_y[|\mathcal{E}(x,y)|/|y|]$ *is non-decreasing in* $d(x)$*.*

(A2) ***Category-level KL is problem-independent:*** *There exist constants* $D_\mathcal{E}, D_\mathcal{C} > 0$ *such that for all problems* x*:*

$$\mathbb{E}[D_{\mathrm{KL}}(\pi_\theta(\cdot \mid x, y_{<t}) \| \pi_{\bar{\theta}}(\cdot \mid x, c, y_{<t})) \mid t \in \mathcal{C}(x,y)] = D_\mathcal{C}, \tag{14}$$

$$\mathbb{E}[D_{\mathrm{KL}}(\pi_\theta(\cdot \mid x, y_{<t}) \| \pi_{\bar{\theta}}(\cdot \mid x, c, y_{<t})) \mid t \in \mathcal{E}(x,y)] = D_\mathcal{E}. \tag{15}$$

(A3) ***Compressible tokens carry strictly larger KL:*** $D_\mathcal{C} > D_\mathcal{E}$*.*

Then the expected normalized compression signal

$$S(x) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} D_{\mathrm{KL}}(\pi_\theta(\cdot \mid x, y_{<t}) \| \pi_{\bar{\theta}}(\cdot \mid x, c, y_{<t})) \right] \tag{16}$$

is non-increasing in $d(x)$*.*

Proof. Decompose the normalized KL into essential and compressible contributions. For a given rollout y:

$$\frac{1}{|y|} \sum_t D_{\mathrm{KL}}(q_t \| p_t) = \underbrace{\frac{|\mathcal{E}|}{|y|} \cdot \bar{D}_\mathcal{E}}_{\text{essential term}} + \underbrace{\frac{|\mathcal{C}|}{|y|} \cdot \bar{D}_\mathcal{C}}_{\text{compressible term}}, \tag{17}$$

where $q_t = \pi_\theta(\cdot \mid x, y_{<t})$, $p_t = \pi_{\bar{\theta}}(\cdot \mid x, c, y_{<t})$, and $\bar{D}_\mathcal{E}, \bar{D}_\mathcal{C}$ denote the average per-token KL on essential and compressible tokens in this rollout, respectively. Taking expectations over y and applying assumption (A2):

$$S(x) = \rho(x) \cdot D_\mathcal{E} + (1 - \rho(x)) \cdot D_\mathcal{C} \tag{18}$$

$$= D_\mathcal{C} - \rho(x) \cdot (D_\mathcal{C} - D_\mathcal{E}). \tag{19}$$

By (A3), $D_\mathcal{C} - D_\mathcal{E} > 0$, so $S(x)$ is a strictly decreasing affine function of $\rho(x)$. Since $\rho(x)$ is non-decreasing in $d(x)$ by (A1), $S(x)$ is non-increasing in $d(x)$.

Quantitatively, for two problems with difficulties $d_1 < d_2$ (hence $\rho(x_1) \leq \rho(x_2)$ by A1):

$$S(x_1) - S(x_2) = (\rho(x_2) - \rho(x_1)) \cdot (D_\mathcal{C} - D_\mathcal{E}) \geq 0. \tag{20}$$

□

Remark 2. *The core assumption is (A1): harder problems have a larger fraction of essential tokens. This is empirically supported by Table 2 (MATH-500: 57–59% compression vs. AIME 2025: ∼35%). Assumption (A2) is a modeling simplification; the proposition should be interpreted as holding for category-averaged KL values. Assumption (A3) does not follow from the reward sign alone—it is a structural assumption motivated by the intuition that compressible positions, where teacher and student distributions diverge most, carry larger full-vocabulary KL.*

A.5 Bounded Forgetting from the Base Model

A central advantage of on-policy self-distillation over off-policy SFT is controlled divergence from the original model. We formalize this through the *conciseness gap*.

Definition 3 (Conciseness gap)**.** The conciseness gap of the base model π_{θ_0} under instruction c on input x is

$$\gamma(x) = d_{\mathrm{TV}}(\pi_{\theta_0}(\cdot \mid x),\ \pi_{\theta_0}(\cdot \mid x, c)). \tag{21}$$

Proposition 3 (Bounded forgetting under on-policy self-distillation)**.** *Consider the first teacher window where $\bar{\theta} = \theta_0$ (frozen teacher). If the converged CRISP loss satisfies $\mathcal{L}(\theta^*) \le \epsilon_{\mathrm{KL}}$, then:*

$$\mathbb{E}_{x\sim\mathcal{D}}[d_{\mathrm{TV}}(\pi_{\theta^*}(\cdot \mid x),\ \pi_{\theta_0}(\cdot \mid x))] \le \sqrt{\frac{\epsilon_{\mathrm{KL}}}{2}} + \mathbb{E}_{x\sim\mathcal{D}}[\gamma(x)]. \tag{22}$$

For subsequent windows with periodic teacher update, the same bound holds with θ_0 replaced by the teacher weights $\bar{\theta}$ at the start of that window. Moreover, $\gamma(x)$ is difficulty-adaptive: for hard problems where the conciseness instruction has little effect, $\gamma(x) \approx 0$, so forgetting is minimal.

Proof. By the triangle inequality for total variation distance:

$$d_{\mathrm{TV}}(\pi_{\theta^*}(\cdot \mid x),\ \pi_{\theta_0}(\cdot \mid x)) \le \underbrace{d_{\mathrm{TV}}(\pi_{\theta^*}(\cdot \mid x),\ \pi_{\theta_0}(\cdot \mid x, c))}_{\text{student–teacher gap}} + \underbrace{d_{\mathrm{TV}}(\pi_{\theta_0}(\cdot \mid x, c),\ \pi_{\theta_0}(\cdot \mid x))}_{\gamma(x)}. \tag{23}$$

The student–teacher gap is bounded via Pinsker's inequality. By Corollary 1, $\mathbb{E}_x[D_{\mathrm{KL}}(\pi_{\theta^*}(\cdot \mid x) \| \pi_{\theta_0}(\cdot \mid x, c))] \le \epsilon_{\mathrm{KL}}$. Applying Pinsker to each x and Jensen's inequality over $x \sim \mathcal{D}$:

$$\mathbb{E}_x[d_{\mathrm{TV}}(\pi_{\theta^*}(\cdot \mid x),\ \pi_{\theta_0}(\cdot \mid x, c))] \le \sqrt{\tfrac{\epsilon_{\mathrm{KL}}}{2}}. \tag{24}$$

Taking expectations on both sides of the triangle inequality yields (22).

For the difficulty-adaptive claim: on hard problems, the conciseness instruction cannot substantially alter the output distribution because most reasoning steps are essential. Thus $\pi_{\theta_0}(\cdot \mid x, c) \approx \pi_{\theta_0}(\cdot \mid x)$, giving $\gamma(x) \approx 0$. □

Remark 3 (Comparison with off-policy SFT)**.** *Standard off-policy SFT minimizes $-\mathbb{E}_{(x,y)\sim\mathcal{D}_T}[\log \pi_\theta(y \mid x)]$ on a fixed teacher dataset $\mathcal{D}_T$. The analogous forgetting decomposition is:*

$$d_{\mathrm{TV}}(\pi_{\theta_{\mathrm{SFT}}}(\cdot \mid x),\ \pi_{\theta_0}(\cdot \mid x)) \le d_{\mathrm{TV}}(\pi_{\theta_{\mathrm{SFT}}}(\cdot \mid x),\ \mathcal{D}_T(\cdot \mid x)) + d_{\mathrm{TV}}(\mathcal{D}_T(\cdot \mid x),\ \pi_{\theta_0}(\cdot \mid x)). \tag{25}$$

The second term measures the distribution mismatch *between the teacher's data and the base model's outputs, which can be substantially larger than the conciseness gap $\gamma(x)$, particularly when the teacher generates qualitatively different reasoning styles. Crucially, off-policy SFT drives $\pi_{\theta_{\mathrm{SFT}}}$ toward $\mathcal{D}_T$ directly, while on-policy distillation generates data from the student's own evolving distribution, inherently limiting divergence from the base model (Shenfeld et al., 2026).*

A.6 Compression Reduces Compounding Error

We provide a simple probabilistic model explaining why shorter reasoning traces can *improve* accuracy.

Proposition 4 (Shorter traces reduce error accumulation)**.** *Suppose each token position independently introduces a reasoning error with probability $p_{\mathrm{err}} \in (0, 1)$, so a trace of length L is correct with probability $\mathrm{Acc}(L) = (1 - p_{\mathrm{err}})^L$. Compressing from L to αL tokens ($\alpha \in (0, 1)$) without increasing the per-token error rate yields an accuracy ratio*

$$\frac{\mathrm{Acc}(\alpha L)}{\mathrm{Acc}(L)} = (1 - p_{\mathrm{err}})^{-(1-\alpha)L} \ge 1 + (1 - \alpha)L \cdot p_{\mathrm{err}}, \tag{26}$$

which grows exponentially in the number of removed tokens. The lower bound follows from $-\ln(1 - p) \ge p$ and $e^u \ge 1 + u$.

Student Prompt $\pi_\theta(\cdot \mid x)$

```
Solve the following math problem step by step. The last line of your response
should be of the form Answer: $Answer (without quotes) where $Answer is the
answer to the problem.

Find all real numbers x such that x^3 - 6x^2 + 11x - 6 = 0.

Remember to put your answer on its own line after "Answer:".
```

Teacher Prompt $\pi_{\bar{\theta}}(\cdot \mid x, c)$

```
Conciseness instruction c: Solve the following math problem concisely and
correctly. Be direct — avoid unnecessary elaboration, redundant steps, or
restating the problem. Focus only on the key reasoning steps needed to reach
the answer.

The last line of your response should be of the form Answer: $Answer (without
quotes) where $Answer is the answer to the problem.

Find all real numbers x such that x^3 - 6x^2 + 11x - 6 = 0.

Remember to put your answer on its own line after "Answer:".
```

Figure 3: **Prompt example for student and teacher policies.** Both policies share the same model parameters but differ in conditioning context. The teacher receives only a *conciseness instruction* c prepended to the problem; no ground-truth answers or reference solutions are provided. This is the key distinction from prior self-distillation work (Shenfeld et al., 2026), where the teacher receives the ground-truth solution as privileged information. The student prompt is the original prompt from the DAPO-17K dataset.

Quantitative interpretation. On MATH-500, CRISP removes $(1-\alpha)L \approx 2{,}750$ tokens ($\alpha \approx 0.41$, $L \approx 4{,}660$). With $p_{\text{err}} = 10^{-4}$, the accuracy ratio is ≥ 1.275 (linear bound) or $\approx e^{0.275} \approx 1.32$ (exponential). The independence assumption is conservative: correlated errors (where one mistake derails subsequent steps) make compression even more beneficial, which helps explain why empirical gains (e.g., 70.0→86.1 on MATH-500) exceed this simple model's predictions.

B Prompt Templates

Figure 3 shows the exact student and teacher prompt templates used in our default setup. Both policies share the same underlying model; the only difference is that the teacher receives a prepended conciseness instruction and no privileged supervision such as a ground-truth answer.

C Ablation Study

C.1 Do Quantitative Reduction Targets Outperform Qualitative Instructions?

Our default teacher simply says "be concise." But what if we gave it a number? Telling the model to "use 50% fewer tokens" seems more precise—surely that would compress harder? We investigate this by comparing four context variants, all trained with the same periodic teacher update (M=50): the static conciseness instruction and soft budget targets at $p \in \{20\%, 50\%, 80\%\}$. The soft budget teacher prompt (Figure 4) replaces the qualitative conciseness instruction with a specific reduction target while keeping all other aspects identical.

Soft Budget Teacher Prompt $\pi_{\bar{\theta}}(\cdot \mid x, c_p)$

Soft budget instruction c_p: Solve the following math problem correctly using p% fewer tokens than you normally would. Be more concise — cut unnecessary elaboration, redundant steps, and verbose explanations while preserving correctness.

The last line of your response should be of the form Answer: $Answer (without quotes) where $Answer is the answer to the problem.

Find all real numbers x such that $x^3 - 6x^2 + 11x - 6 = 0$.

Remember to put your answer on its own line after "Answer:".

Figure 4: **Soft budget teacher prompt.** Unlike the qualitative conciseness instruction in Figure 3, the soft budget variant specifies a quantitative reduction target $p \in \{20, 50, 80\}$. The student prompt remains unchanged from Figure 3.

Table 4: **Qualitative conciseness instructions outperform quantitative reduction targets on accuracy (30K token budget).** All variants use periodic teacher update (M=50) at step 100. Soft budgets achieve higher compression but substantially lower accuracy than the concise instruction, particularly on competition-level benchmarks. Accuracy (Acc, %), token reduction (Red., %), and accuracy change vs. base model (ΔAcc, pp).

	MATH-500			AIME 2024			AIME 2025		
Context	Acc	Red.	ΔAcc	Acc	Red.	ΔAcc	Acc	Red.	ΔAcc
Qwen3-8B									
Concise	**86.6**	58.8%	**+8.9**	69.6	35.4%	−2.9	**57.1**	35.7%	**−5.4**
Soft (p=20%)	86.2	60.1%	+8.6	**70.8**	39.5%	**−1.7**	52.9	33.7%	−9.6
Soft (p=50%)	85.9	60.4%	+8.2	63.8	36.3%	−8.8	52.1	36.2%	−10.4
Soft (p=80%)	84.1	**63.8%**	+6.5	67.9	**41.5%**	−4.6	49.6	**39.5%**	−12.9
Qwen3-14B									
Concise	**86.1**	56.5%	**+16.1**	**76.3**	41.0%	**+10.5**	**61.7**	35.2%	**−5.4**
Soft (p=20%)	80.7	67.8%	+10.8	67.1	47.8%	+1.3	57.9	45.8%	−9.2
Soft (p=50%)	80.7	67.2%	+10.8	67.1	49.7%	+1.3	57.5	48.8%	−9.6
Soft (p=80%)	79.8	**68.9%**	+9.9	68.3	**50.8%**	+2.5	54.6	**50.7%**	−12.5

Table 4 reveals a clear **compression–accuracy tradeoff across context variants**: soft budgets achieve higher compression but substantially lower accuracy than the qualitative concise instruction.

Soft budgets compress more aggressively but sacrifice accuracy. On MATH-500, p=80% achieves **63.8%** token reduction for Qwen3-8B (versus the concise instruction's 58.8%), but accuracy drops from 86.6% to 84.1%. The gap widens dramatically on competition-level benchmarks: for Qwen3-14B on AIME 2024, the concise instruction achieves 76.3% accuracy while all soft budget variants cluster around 67–68%. The concise instruction achieves the best accuracy on 5 of 6 model–benchmark combinations. The sole exception is Qwen3-8B on AIME 2024, where p=20% reaches 70.8% versus 69.6% for the concise instruction, a difference within evaluation noise.

Compression monotonically increases with p, but accuracy does not. For Qwen3-14B, token reduction on AIME 2024 increases with the target: p=20% achieves 47.8%, p=50% achieves 49.7%, and p=80% achieves 50.8%. However, the best soft-budget accuracy on AIME 2024 comes from p=80% (68.3%), not p=20% (67.1%), suggesting that the relationship between reduction target and accuracy is non-monotonic.

These results recommend the **qualitative concise instruction as the default**: it achieves the best accuracy, compresses substantially (57–59% on MATH-500), and—crucially—remains stable under extended training. The lesson: vague instructions make better teachers than precise ones.

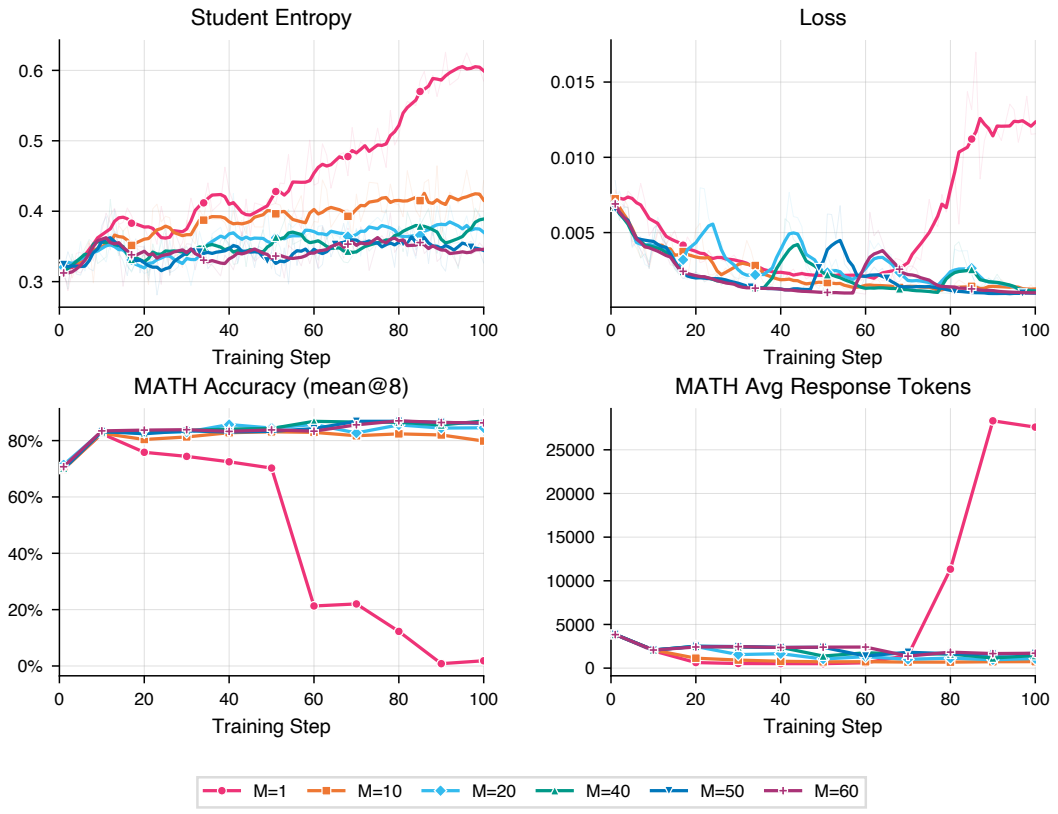

Figure 5: **Teacher update interval M controls the stability–compression trade-off.** Accuracy (left) and output entropy (right) over 100 training steps for Qwen3-14B on MATH-500 with varying M. M=1 (updating every step) causes entropy explosion and accuracy collapse to ∼2% by step 100, consistent with the instability observed by Shenfeld et al. (2026). $M \in \{40, 50, 60\}$ produce stable trajectories reaching ∼86–87% accuracy. M=10 peaks early then degrades, while M=20 remains competitive but shows mild entropy drift. All experiments use the qualitative concise instruction.

C.2 How Sensitive Is Compression to the Teacher Update Interval?

The teacher update interval M (Eq. 2) controls how frequently the teacher weights are synchronized with the student. A larger M provides a more stable distillation target but limits progressive compression; a smaller M pushes compression further but risks training instability when the teacher changes too rapidly. We sweep $M \in \{1, 10, 20, 40, 50, 60\}$ using Qwen3-14B with the qualitative concise instruction on MATH-500.

Figure 5 reveals three distinct regimes:

M=1 **is catastrophically unstable.** Updating the teacher after every gradient step causes entropy to explode from ∼0.32 to ∼0.58 and accuracy to collapse from a peak of ∼82% (step 10) to ∼2% (step 100). This mirrors the finding of Shenfeld et al. (2026) that overly aggressive teacher updates create a moving target problem: the student chases a teacher that is itself changing in response to the student's updates, leading to a positive feedback loop of increasingly degenerate outputs.

$M \in \{40, 50, 60\}$ **form a stable plateau.** These intervals produce similar accuracy trajectories, all reaching ∼86–87% by step 100 with entropy remaining stable around ∼0.33–0.39. The method is robust to the exact choice of M within this range: the teacher remains stable long enough for the student to meaningfully converge toward the current target before the next refresh.

M=10 **degrades;** M=20 **is borderline.** M=10 peaks at ∼83% accuracy around step 50 but declines to ∼80% by step 100, with entropy drifting up to ∼0.44. M=20 performs better (84.5% at step 100) but still trails the $M \geq 40$ regime by 2–3 percentage points.

Based on these results, we use M=50 for all other experiments in this paper, as it sits comfortably in the stable plateau while allowing progressive compression through periodic teacher refresh.

D Survey of Reasoning Compression Methods

Table 5 summarizes 19 reasoning compression methods along four axes. This survey motivates the design of CRISP by revealing the pervasive dependence on ground-truth answers and the rarity of difficulty-adaptive methods.

Table 5: Survey of 19 reasoning compression methods. LP = Length Penalty in reward; DD = Difficulty-Dependent; CA = Correct Answer required; HB = Hard Budget.

Method	LP	DD	CA	HB	Approach
L1	✓		✓	✓	RL
DiPO	✓	✓	✓		RL
TRAAC	✓	✓	✓		RL
DIET	✓	✓	✓		RL
DLER	✓	✓	✓		RL
Leash	✓		✓		RL
ORION	✓		✓		RL
AdaptThink		✓	✓		RL
SEER			✓		SFT
TokenSkip			✓	✓	SFT
V-Skip			✓		SFT
S3-CoT					Steering
DAP/LiteCoT		✓	✓		SFT
Extra-CoT			✓		SFT
CtrlCoT			✓	✓	SFT
Chain of Draft					Prompt
TrimR					Inference
NoWait					Inference
FlowSteer					Inference
CRISP (Ours)		✓			Self-distill

E Extended Results Across Token Budgets

E.1 Results Under 8K Token Budget

Table 6 reports results under the efficient-serving token budget of 8,192 tokens. Because the base model frequently produces responses exceeding this limit on harder benchmarks (AIME 2024/2025), truncation disproportionately affects the verbose baseline, making the compression gains appear even larger. We include these results as a practical reference for deployment scenarios where strict token budgets are enforced.

Truncation explains the amplified accuracy gains. Comparing Tables 2 and 6 reveals a striking pattern: the accuracy gap between CRISP and the base model is far larger under the 8K budget than under 30K. For example, Qwen3-8B on AIME 2024 shows +29.6 pp under 8K (25.0→54.6) but −2.9 pp under 30K (72.5→69.6). The mechanism is straightforward: under the 8K cap, the verbose base model's responses are frequently truncated before reaching the answer, causing catastrophic accuracy loss (the base model drops from 72.5% to 25.0% on AIME 2024 when the budget shrinks from 30K to 8K). CRISP's compressed responses largely fit within the 8K window, avoiding this truncation penalty entirely. The lesson is

practical: **under strict serving budgets, reasoning compression is not merely an efficiency tool—it is essential for preserving accuracy**.

Table 6: **Self-distillation results under the 8,192-token budget.** Same setup as Table 2 but with max response length capped at 8,192 tokens, representative of efficient serving constraints. Accuracy (Acc, mean over 8 samples, %), average reasoning token length (Len), and token reduction (Red., %).

	MATH-500			AIME 2024			AIME 2025		
Method	Acc	Len	Red.	Acc	Len	Red.	Acc	Len	Red.
Qwen3-8B									
Base Model	69.4	3,860	—	25.0	7,597	—	18.8	7,661	—
Concise prompt	78.1	2,514	34.9%	37.9	6,870	9.6%	29.2	7,140	6.8%
CRISP	**85.1**	1,199	**68.9%**	**54.6**	5,114	**32.7%**	**36.3**	5,658	**26.2%**
Qwen3-14B									
Base Model	64.3	3,285	—	27.1	7,345	—	19.6	7,471	—
Concise prompt	81.8	2,059	37.3%	45.4	6,483	11.7%	30.8	6,852	8.3%
CRISP	**85.5**	1,066	**67.6%**	**57.5**	4,599	**37.4%**	**39.2**	5,484	**26.6%**

E.2 Extended Training Under 30K Token Budget

Table 7 shows performance at training steps 100 and 200 under the 30K token budget, illustrating the accuracy–compression trade-off as training progresses beyond the sweet spot. At step 100 (our default checkpoint), both models achieve strong compression with accuracy improvements on MATH-500. Continuing to step 200 roughly doubles token reduction on AIME benchmarks (from ∼35–41% to ∼51–53%) but at the cost of AIME accuracy, particularly for the harder AIME 2025. MATH-500 accuracy remains robust throughout, suggesting that compression on easier benchmarks is more sustainable.

Practical recommendation: step 100 is the sweet spot. Based on these results, we recommend step 100 (roughly 3,200 training examples) as the default checkpoint. It achieves 57–59% compression on MATH-500 with accuracy gains of 9–16 pp, while AIME accuracy degrades by at most 5 pp. Continuing to step 200 yields diminishing compression returns on MATH-500 (+13 pp additional reduction for Qwen3-8B) but substantial accuracy losses on competition benchmarks (-7.5 pp on AIME 2024, -10.4 pp on AIME 2025 for 8B).

Difficulty-dependent over-compression. The divergent behavior across benchmarks at step 200 is consistent with the difficulty-adaptive compression analysis in Section A.4. On easy benchmarks like MATH-500, most tokens are compressible (the essential fraction $\rho(x)$ is small), so further compression removes genuinely redundant tokens and accuracy is preserved. On hard benchmarks like AIME 2025, $\rho(x)$ is already high at step 100; continued training pushes compression into essential tokens, degrading accuracy. This provides a practical corollary to Proposition 2: **the optimal training duration is itself difficulty-dependent**, and a single checkpoint must trade off easy-problem compression against hard-problem preservation.

F Training and Implementation Details

Technical setup. All experiments are conducted on a single node equipped with eight NVIDIA H200 GPUs. Our implementation is built on top of the `verl` library Sheng et al. (2025), which provides a HybridEngine for efficient actor–rollout–reference model co-location. We use PyTorch Fully Sharded Data Parallel (FSDP) for distributed training with parameter and optimizer offloading to CPU, and SGLang Zheng et al. (2024) for batched rollout generation. Sequence parallelism (Ulysses, degree 4) is enabled during training to handle long sequences efficiently, while tensor parallelism (degree 2) is used for inference. Mixed-precision training is performed in `bfloat16`, and gradient checkpointing is enabled to reduce peak memory usage.

Table 7: **Extended training results under the 30K token budget.** Performance at step 100 (default) and step 200, showing the accuracy–compression trade-off with continued training. Accuracy (Acc, mean over 8 samples, %), average reasoning token length (Len), and token reduction vs. base model (Red., %).

	MATH-500			AIME 2024			AIME 2025		
Method	Acc	Len	Red.	Acc	Len	Red.	Acc	Len	Red.
Qwen3-8B									
Base Model	77.7	4,661	—	72.5	14,170	—	62.5	16,682	—
CRISP (step 100)	**86.6**	1,921	58.8%	**69.6**	9,152	35.4%	**57.1**	10,726	35.7%
CRISP (step 200)	85.1	1,305	**72.0%**	62.1	6,902	**51.3%**	46.7	8,146	**51.2%**
Qwen3-14B									
Base Model	70.0	3,872	—	65.8	12,844	—	67.1	15,642	—
CRISP (step 100)	86.1	1,686	56.5%	**76.3**	7,577	41.0%	**61.7**	10,137	35.2%
CRISP (step 200)	**86.2**	1,191	**69.2%**	66.3	6,089	**52.6%**	53.8	7,496	**52.1%**

Training data. Our training data is derived from DAPO-Math-17k Yu et al. (2025), a deduplicated set of ∼17,000 competition-level math problems. We randomly split the dataset into 80% training (∼13,600 prompts) and 20% validation (∼3,400 prompts) with a fixed seed for reproducibility across all configurations. For each problem, we construct a *student prompt* (the original question) and a *teacher prompt* (the question prepended with a conciseness instruction).

Training procedure. At each training step, the student model generates a response from the student prompt via SGLang sampling (temperature 1.0, top-p 1.0). We then perform a single gradient update minimizing the reverse KL divergence between student and teacher logit distributions over the student's own generated tokens. All student rollouts are used for training regardless of correctness; no filtering is applied. Both teacher and student forward passes are performed for each micro-batch with chunked logit processing (chunk size 256 tokens) to bound peak GPU memory; teacher logits are progressively freed after each chunk.

Hyperparameters. Table 8 summarizes the full configuration, which is shared across all models and instruction variants.

Evaluation. We evaluate every 10 steps on three held-out math benchmarks: MATH-500 Hendrycks et al. (2021), AIME 2024, and AIME 2025. For each benchmark, we generate 8 responses per problem with temperature 0.6, top-$p = 0.95$, and top-$k = 20$, and report mean accuracy (fraction of correct samples averaged over problems) and average response token count. Correctness is determined by extracting the final answer and comparing against the ground truth using the symbolic and numeric equivalence checker from the `verl` library (Sheng et al., 2025).[3]

G Alternative Teacher Parameterizations

The main paper uses a *periodic teacher update* ($\bar{\theta} \leftarrow \theta$ every M steps) that balances progressive compression with training stability. Here we discuss the design space of teacher parameterizations, from the most conservative to the most aggressive. An empirical comparison across update intervals $M \in \{1, 10, 20, 40, 50, 60\}$ is provided in Section C.2.

Frozen teacher ($M = \infty$). The simplest variant fixes $\bar{\theta} = \theta_0$ for the entire training run. This provides a stable, non-shifting compression target and allows teacher prefill to be precomputed once. However, the frozen teacher becomes an increasingly weak compression oracle as the student improves, limiting the maximum achievable compression.

[3] https://github.com/verl-project/verl/blob/main/verl/utils/reward_score/math_dapo.py

Parameter	Value
General	
Models	Qwen3-8B, Qwen3-14B
Loss function	Reverse KL: $\mathrm{KL}(\pi_{\text{student}} \| \pi_{\text{teacher}})$
Teacher	Periodic update (M=50 steps)
Data	
Training prompts	∼13,600 (from DAPO-Math-17k)
Validation prompts	∼3,400
Max prompt length	1,024 tokens
Max response length	8,192 tokens (training)
Generation (student rollout)	
Inference engine	SGLang
Temperature	1.0
Top-p	1.0
Rollouts per prompt	1
Max generation tokens	9,216
Evaluation	
Temperature	0.6
Top-p	0.95
Top-k	20
Rollouts per prompt	8
Max generation tokens	30,000
Eval frequency	Every 10 steps
Training	
Optimizer	AdamW
Learning rate	1×10^{-6} (constant)
Weight decay	0.01
Gradient clipping	1.0 (max norm)
Global batch size	32
Micro-batch size per GPU	2
Epochs	1
Precision	bfloat16
Infrastructure	
GPUs	8× NVIDIA H200
Tensor parallelism (inference)	2
Sequence parallelism (training)	Ulysses, degree 4
FSDP parameter offload	Enabled
FSDP optimizer offload	Enabled
Gradient checkpointing	Enabled

Table 8: Hyperparameters for CRISP. The same configuration is used across all models and instruction variants.

Periodic teacher (our default, M=50). Setting a finite update interval M (Algorithm 1) enables progressive compression: after each refresh, the updated teacher, having already internalized compression from the previous round, produces even more concise traces under instruction c, providing a stronger compression signal. The discrete refresh avoids continuous co-adaptation while still allowing compression to deepen over training. Our ablation (Section C.2) shows that $M \in \{40, 50, 60\}$ form a stable plateau with nearly identical training dynamics, confirming robustness to the exact interval within this range.

EMA teacher. The teacher parameters are an exponential moving average of the student, updated after each gradient step:

$$\bar{\theta} \leftarrow \alpha\bar{\theta} + (1-\alpha)\theta, \quad \alpha \in [0.99, 0.999]. \tag{27}$$

This provides a smooth, continuous version of progressive compression. With moderate decay ($\alpha = 0.995$), it can yield additional compression beyond the frozen teacher but requires careful monitoring for collapse.

Stop-gradient concurrent teacher (M=1). The teacher uses the *same* parameters θ as the student, with stop-gradient during the backward pass. This provides the most aggressive progressive compression but carries the highest risk of *progressive compression collapse*: as the student becomes more concise, the teacher also becomes more concise, creating a positive feedback loop that can drive output length toward degenerate short sequences. Our ablation confirms this prediction empirically: M=1 causes entropy explosion and accuracy collapse (Figure 5), consistent with the moving-target instability identified by Shenfeld et al. (2026).

H Token Reduction and Accuracy over Training

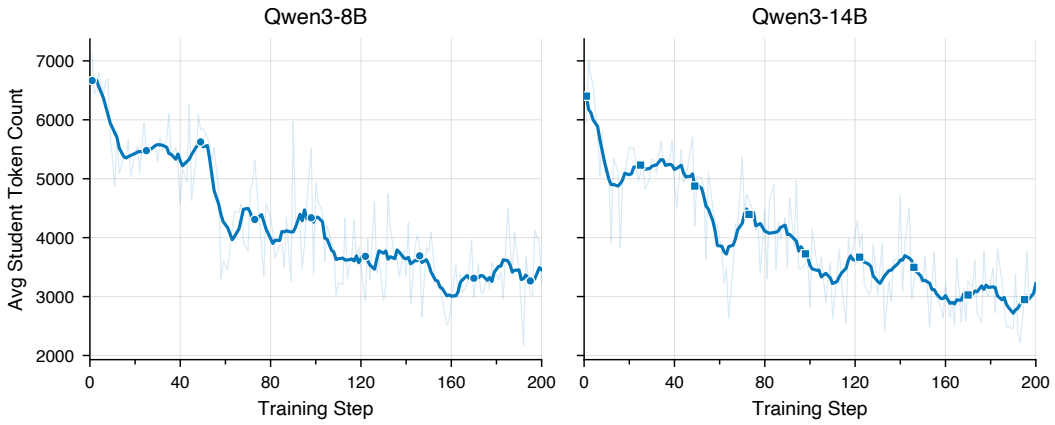

Figure 6: **Average response token count over 200 training steps** for Qwen3-8B and Qwen3-14B using the qualitative concise instruction with periodic teacher update (M=50). Token count decreases rapidly in the first ∼80 steps before plateauing around 3000–3500 tokens. Further compression between steps 100 and 200 is limited, indicating that most compression is learned early and additional training yields diminishing returns.

Figure 6 illustrates a key practical advantage of CRISP: **compression is achieved early and remains stable over extended training**.

For both Qwen3-8B and Qwen3-14B, the bulk of token reduction occurs within the first ∼80 training steps, after which average token count plateaus around 3000–3500 tokens. Extending training to 200 steps yields only marginal additional compression beyond step 100, indicating that the dense, per-token KL signal enables rapid convergence to a stable compression level. This means practitioners can achieve nearly full compression benefit with modest compute budgets. Crucially, within each M-step teacher window the plateau is *stable*: token length does not continue to decrease, confirming that the fixed teacher within a window acts as a stable attractor (cf. the collapse risk with M=1 in Section C.2).

Figure 7 confirms that accuracy improvements track compression dynamics. On MATH-500, both models show steady accuracy gains that co-occur with the token reduction in Figure 6, reinforcing the finding that compression *causes* accuracy gains rather than merely coinciding with them. For AIME 2024 and AIME 2025, the small sample sizes (30 problems each) introduce substantial evaluation variance, making step-by-step trends less reliable; nevertheless, accuracy remains stable or slightly improving throughout training, showing no evidence of an accuracy–compression trade-off. The high variance on competition

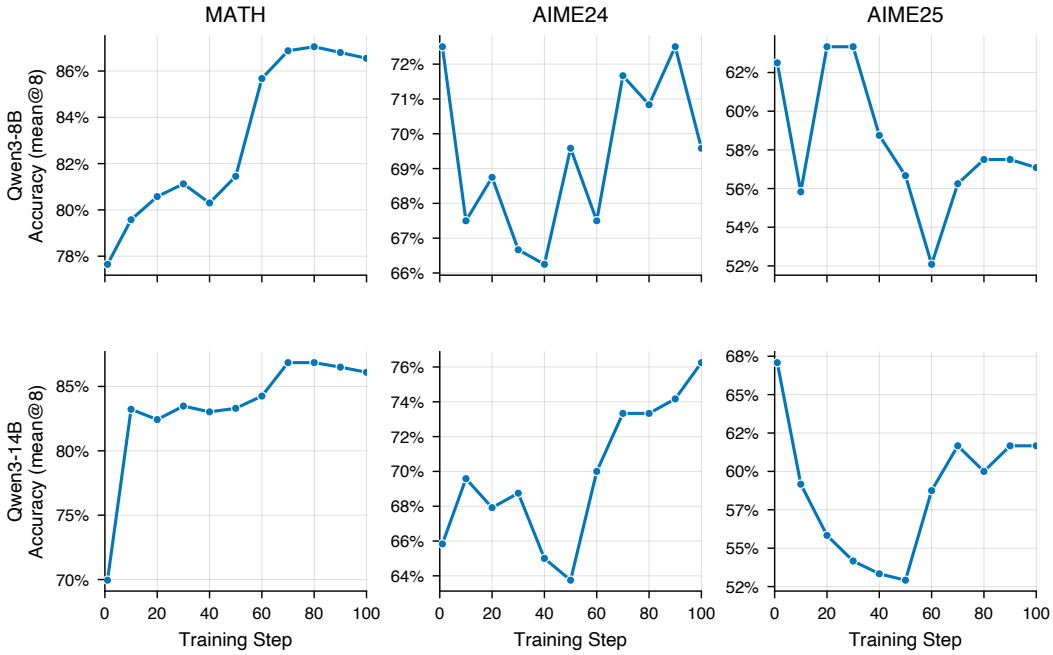

Figure 7: **Validation accuracy (mean@8) over training steps** for Qwen3-8B and Qwen3-14B using the qualitative concise instruction with periodic teacher update (M=50), evaluated on MATH-500, AIME 2024, and AIME 2025. MATH-500 accuracy improves steadily for both models, rising from ∼78% to ∼87% (8B) and ∼70% to ∼87% (14B). AIME 2024 and AIME 2025 results exhibit substantially larger variance due to their small sample sizes (30 problems each), though the overall trend remains stable or slightly improving.

benchmarks underscores the importance of using larger evaluation sets (e.g., MATH-500) for monitoring training dynamics.

H.1 Entropy Stability Throughout Training

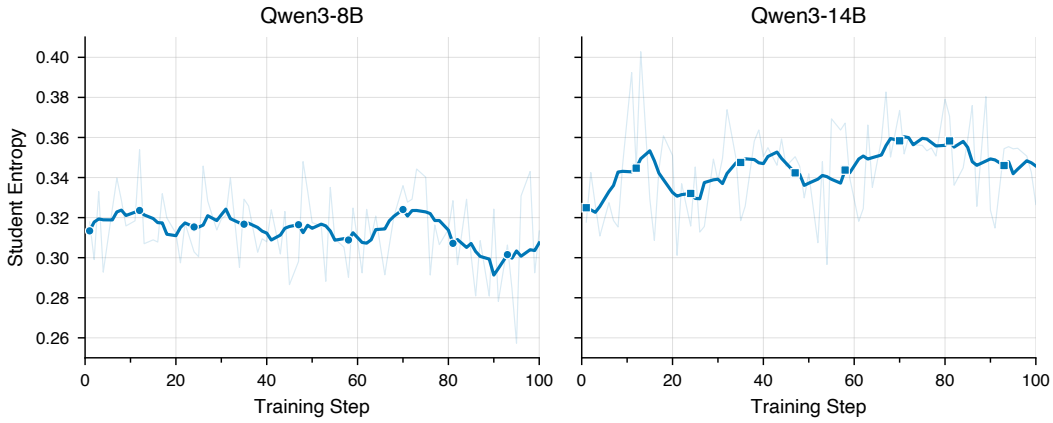

Figure 8: **Self-distillation preserves model entropy throughout training.** Average per-token entropy of the student model over training steps for Qwen3-8B (left) and Qwen3-14B (right) using the concise instruction. Unlike RL with length penalties, which drives entropy toward collapse (Liu et al., 2025; Cui et al., 2025), CRISP maintains stable entropy: the model learns to be concise without losing its exploratory capacity.

Figure 8 shows that CRISP maintains stable per-token entropy throughout training, in stark contrast to RL methods with length penalties, which drive entropy toward collapse (Liu et al., 2025; Cui et al., 2025). This follows from the mode-seeking property of reverse KL (§3.2): the student is penalized for placing mass where the teacher assigns low probability, but *not* for maintaining mass where the teacher is also uncertain. RL length penalties, by contrast, reward shorter outputs regardless of token informativeness, systematically suppressing high-entropy "exploratory" tokens ("Wait," "Alternatively," "Let me reconsider…") that are critical for solving hard problems (Wang et al., 2025b).

I Effect of KL Divergence Direction in CRISP

I.1 Background and Motivation

The CRISP distillation loss aligns the live student p_S with the teacher p_T, a periodically frozen snapshot of the student itself. A natural question is which direction of KL divergence to use:

- **Reverse KL** (our choice): $\mathrm{KL}(p_S \| p_T) = \sum_v p_S(v)\big[\log p_S(v) - \log p_T(v)\big]$. The gradient w.r.t. student parameters is weighted by the *student's own* distribution $p_S(v)$: the student updates only in regions where it currently generates, providing built-in self-regularization against abrupt distribution shifts.
- **Forward KL** (baseline): $\mathrm{KL}(p_T \| p_S) = \sum_v p_T(v)\big[\log p_T(v) - \log p_S(v)\big]$. The gradient is weighted by the *teacher's* distribution $p_T(v)$, fully decoupled from the student's current state.

Forward KL is sometimes preferred in offline distillation because the teacher is a fixed, authoritative external model—its distribution is a reliable signal to track exactly. In CRISP, however, the teacher is not external: it is a stale copy of the student, refreshed every M steps. We argue that this makes forward KL structurally ill-suited for CRISP: because the gradient is scaled by p_T rather than p_S, every teacher refresh injects a large, unconstrained update whose magnitude is independent of how far the student has drifted. As we show below, this produces cascading instability that worsens with each successive refresh.

I.2 Experimental Setup

We compare reverse and forward KL on Qwen3-8B and Qwen3-14B with exactly the same setup as in Table 2, where teacher is updated every 50 steps. Validation pass@1 (8 samples) is measured on MATH-500, AIME 2024, and AIME 2025 every 10 steps.

I.3 Results

Figure 9 shows a stark contrast. Reverse KL rises quickly in the first 50–70 steps and holds a stable plateau through all four teacher refreshes with no visible regression. Forward KL tracks comparably in the first interval—it is even marginally ahead on MATH at step 70—but collapses within 10 steps of every subsequent refresh before partially recovering. The saw-tooth deepens with each cycle: on Qwen3-14B the trough relative to reverse KL grows from $\sim$15% on AIME 2024 after the step-100 refresh, to $>$ 19% after step 150, reaching a $>$ 23% gap by step 190. The same pattern is visible but milder on Qwen3-8B.

Figure 10 reveals a parallel instability: forward KL compresses responses more aggressively, with length drops synchronized to the same teacher-update boundaries. On Qwen3-14B the gap widens to $\sim$500 tokens by step 190 (1,229 vs. 1,766, a 30% shortfall). On hard reasoning benchmarks like AIME, this truncation of thinking chains is both a symptom and a driver of the accuracy degradation.

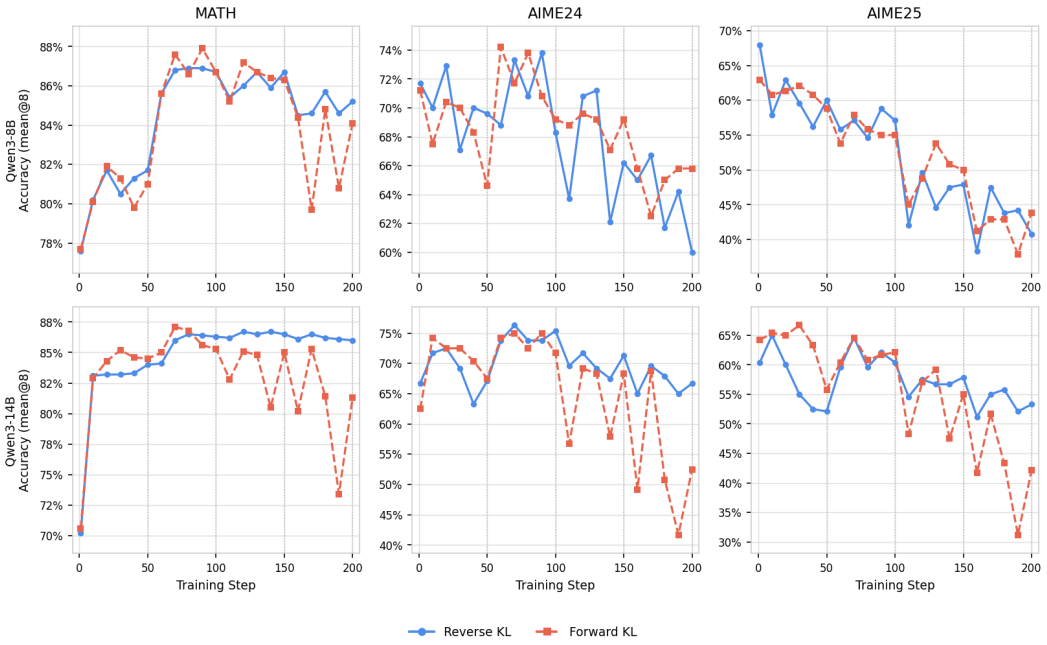

Figure 9: Validation accuracy of reverse KL (solid blue) and forward KL (dashed coral) on Qwen3-8B (top) and Qwen3-14B (bottom). Dotted verticals mark teacher-update steps. Reverse KL holds a stable plateau; forward KL exhibits a saw-tooth that deepens with each refresh.

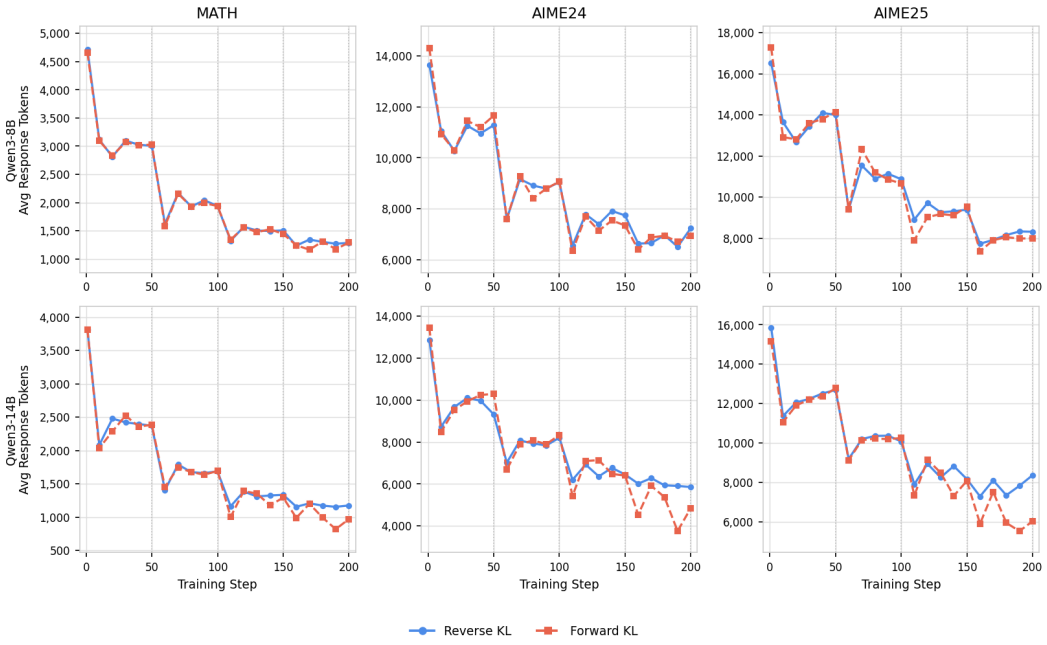

Figure 10: Mean response length on each validation set. Forward KL compresses responses more aggressively, with steeper drops synchronized with each teacher-update boundary.

I.4 Mechanism and Conclusion

The instability follows directly from the gradient structure. Under forward KL, $\nabla_{\log p_S} \mathrm{KL}(p_T \| p_S) = -p_T(v)$: updates are scaled by the teacher's confidence regardless of the student's current state. Each refreshed teacher is slightly more length-compressed than the last, so the distributional shock at each refresh grows, explaining the escalating saw-tooth amplitude. Under reverse KL, the gradient takes the form $\nabla_\theta \mathrm{KL}(p_S \| p_T) = \mathbb{E}_{v \sim p_S}\left[\nabla_\theta \log p_S(v) \left(\log p_S(v) - \log p_T(v) + 1\right)\right]$, so updates are explicitly weighted by the student's own distribution p_S; this provides natural self-regularization. Because the student already covers the teacher's high-probability modes, each refresh requires only a small incremental adjustment.

Reverse KL is the appropriate divergence for iterative on-policy self-distillation. All CRISP experiments in this paper use reverse KL accordingly.

J Deep Planning Agentic Task

Beyond mathematical reasoning, we evaluate whether our compressed models retain their capability on complex, multi-step agentic tasks that require tool calling, constraint satisfaction, and long-horizon planning. We use the **DeepPlanning** benchmark (Zhang et al., 2026), which evaluates LLM agents across two domains:

Travel Planning. The agent is given a natural-language travel request (e.g., "Plan a 3-day trip to Beijing for two people with a $2,000 budget") and must produce a complete itinerary by issuing tool calls to query flights, trains, hotels, restaurants, and attractions. The benchmark contains 240 test cases (120 Chinese + 120 English). Evaluation checks both *hard constraints* (budget, dates, party size) and *commonsense constraints* (route consistency, business hours, activity diversity) across eight dimensions. We report the **composite score**—a weighted average of commonsense and personalization scores.

Shopping Planning. The agent must fulfill a multi-item shopping request (e.g., "Find running shoes under $100, a matching sports watch, and apply any available coupons") by navigating a simulated e-commerce environment with tools for product search, filtering, cart management, and coupon application. The benchmark contains 150 test cases across three difficulty levels. We report the **match rate**—the fraction of expected products correctly placed in the cart.

Setup. We evaluate Qwen3-14B checkpoints from our training (teacher update frequency M=40, reverse KL loss) on 100 travel planning samples and evalaute at every 10 steps. Each checkpoint is served via vLLM and paired with the DeepPlanning function-calling agent, which iteratively invokes the LLM and executes tool calls until a final plan is produced (up to 400 LLM calls per case). We run the base model (step 0) 10 times to establish a variance estimate.

Results. Figure 11 shows the trade-off between response length and planning quality. CRISP training progressively compresses average LLM response tokens: shopping tokens decrease from 12.5k (base) to 6.2k (step 120), a **51% reduction**; travel tokens decrease from 5.3k to 3.1k, a **42% reduction**. Despite this substantial compression, planning accuracy remains largely preserved—shopping match rate stays within the baseline variance through step 70 (25.8% vs. 24.2% base), and travel composite score is stable across all checkpoints (17.0–19.1% vs. 17.7% base). This demonstrates that our reasoning compression method transfers beyond mathematical reasoning to multi-step agentic planning, reducing token consumption without degrading the model's ability to orchestrate complex tool-calling sequences.

Shopping quality degrades after step 70. While travel planning quality remains stable throughout training, shopping match rate begins to decline beyond step 70. We hypothesize that this reflects a difference in the reasoning structure of the two tasks: travel planning

relies on a fixed sequence of tool calls (search → book → verify) where the reasoning overhead is largely redundant narration, whereas shopping planning requires more adaptive, branching logic (compare prices, evaluate coupons, backtrack). Compressing beyond a certain threshold removes not just redundant tokens but also the exploratory reasoning needed for these adaptive decisions. This suggests that the safe compression frontier is task-dependent, aligning with the difficulty-adaptive analysis in Section A.4.

Absolute scores reflect benchmark difficulty, not model weakness. The travel composite scores (17–19%) and shopping match rates (∼25%) may appear low in isolation. These numbers reflect the inherent difficulty of the DeepPlanning benchmark, which evaluates multi-step tool-calling agents on tasks requiring dozens of correctly sequenced API calls with hard constraint satisfaction. For reference, Zhang et al. (2026) report that even frontier models (GPT-4o, Claude 3.5 Sonnet) achieve modest scores on these tasks. The key observation is not the absolute level but the *stability* of scores under compression: CRISP reduces token consumption by 42–51% without degrading planning quality relative to the uncompressed base model.

Limitations. This transfer experiment evaluates only Qwen3-14B; extending to Qwen3-8B and other model families would strengthen the generalization claim. We also do not compare against a concise-prompt-only baseline on DeepPlanning, which would isolate the contribution of CRISP training beyond pure prompting. We leave these extensions for future work.

K Qualitative Examples

Figure 12 provides side-by-side comparisons of *full model outputs* from the base Qwen3-8B model and the CRISP-trained model on three MATH-500 problems of increasing difficulty. Each output consists of hidden reasoning (inside `<think>...</think>` tags) followed by the visible answer presented to the user. The base model exhibits two distinct sources of token overhead:

- **Redundancy within reasoning:** Self-doubt ("Wait," "Let me check..."), re-derivation via alternative methods, and numerical verification of already-established results inside `<think>`.
- **Redundancy between reasoning and visible answer:** The base model produces a fully formatted, step-by-step solution *after* `</think>` that essentially repeats the entire derivation from the hidden reasoning block.

CRISP eliminates both: it compresses the hidden reasoning to core logical steps and produces only the final answer as visible output. The three examples also illustrate difficulty-adaptive compression (Section 3.3):

- **Easy problem (84% reduction):** The algebra word problem admits a short, direct derivation. The base model re-derives via alternative methods and verifies numerically inside `<think>`, then writes a full formatted solution after `</think>`. CRISP retains only the single derivation and outputs the answer directly.
- **Moderate problem (56% reduction):** The number theory problem requires a genuine insight ($10^k \equiv 10 \pmod{18}$). The base model discovers this but repeatedly verifies it, cross-checks via CRT, and writes a formatted step-by-step solution after `</think>`. CRISP discovers the same insight, applies it once with a brief CRT check, and outputs only the answer.
- **Hard problem (52% reduction):** The product-simplification problem requires the Sophie Germain identity and a telescoping argument. The base model arrives at the correct answer but only after extensive numerical exploration, repeated algebraic verification, and an attempt to factor the final result; after `</think>`, it reproduces the full four-step derivation. CRISP applies the identity immediately, identifies the telescoping pattern, and outputs only the answer.

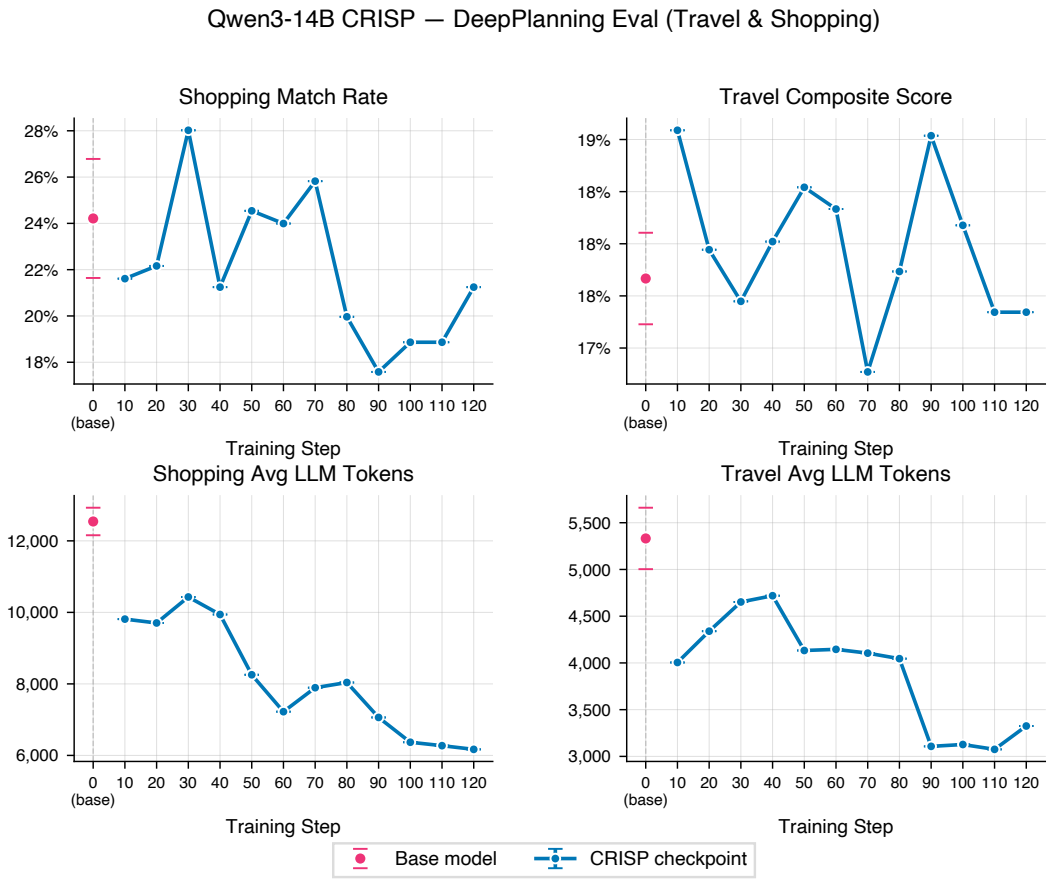

Figure 11: Mean response length on training set travel planning and validation set shopping planning. *Shopping match rate* measures the fraction of ground-truth products correctly placed in the agent's cart (higher is better). *Travel composite score* is the average of a commonsense score (route consistency, time feasibility, business hours, etc., across 8 dimensions) and a personalization score (satisfaction of user-specified hard constraints such as budget, dates, and party size), averaged over Chinese and English test sets (higher is better).

Problem 1 (Algebra): *Ten treeks weigh as much as three squigs and one goolee. Two treeks and one goolee are equal in weight to one squig. The combined weight of how many treeks equals one squig?*

Base model (1,354 tokens)

<think>
...I need to figure out how many treeks weigh the same as one squig....
10T = 3S + G (first equation)
2T + G = S (second equation)
From the second equation: G = S - 2T. Substitute:
10T = 3S + (S - 2T) ⇒ 12T = 4S ⇒ **3T = S.**
Wait, so S equals 3T? ...Let me check my steps again...
Alternatively, let's use substitution. Let me express everything in terms of T... 10T = 6T + 4G ⇒ T = G. Which again gives S = 3T. So same result.
So, if T = G, then...9T = 3S ⇒ 3T = S. Yep, same thing.
Wait, but let me check with actual numbers... Suppose a treek is 1 kg. Then squig is 3 kg, goolee is 1 kg. Check first statement: 10 = 10. Correct. Second: 3 = 3. Correct.
</think>

To determine how many treeks weigh the same as one squig, we are given...
Step 1: Express G in terms of S and T: $G = S - 2T$.
Step 2: Substituting into $10T = 3S + G$...$10T = 4S - 2T$.
Step 3: Add $2T$...$12T = 4S \Rightarrow 3T = S$.
This means that **one squig weighs the same as 3 treeks**.
Answer: **3** ✓

CRISP (221 tokens, 84% reduction)

<think>
The problem gives two equations. 10 treeks = 3 squigs + 1 goolee. 2 treeks + 1 goolee = 1 squig.
Equation 1: 10T = 3S + G
Equation 2: 2T + G = S
From Equation 2, solve for G: G = S - 2T. Substitute into Equation 1:
10T = 3S + (S - 2T)
10T = 4S - 2T
12T = 4S
Divide both sides by 4: **3T = S**
So, 1 squig equals 3 treeks. Therefore, the answer is 3.
</think>

Answer: **3** ✓

Problem 2 (Number Theory): *What integer n satisfies $0 \leq n < 18$ and $n \equiv -11213141 \pmod{18}$?*

Base model (3,979 tokens)

<think>
...I need to find $n \equiv -11213141 \pmod{18}$...
Since 18 is 2×9, maybe I can use the Chinese Remainder Theorem? Wait, maybe that's overcomplicating.
$10^k \equiv 10 \pmod{18}$ for $k \geq 1$. Let me check: $10^1 = 10$, $10^2 = 100 \equiv 10$, $10^3 \equiv 10$...Yes.
Digits: 1,1,2,1,3,1,4,1. $1 + 10 \times (4+1+3+1+2+1+1) = 131$.
131 mod 18 = 5. Let me verify: 18×7=126, 131−126=5. Yes.
So $-11213141 \equiv -5 \equiv 13 \pmod{18}$.
Let me verify by another method...CRT: mod 2 gives 1, mod 9: digit sum $14 \equiv 5$. Numbers 0–17 that are 4 mod 9: 4, 13. 13 is odd. So $n = 13$.
All methods point to 13.
</think>

Step 1: We compute 11213141 mod 18, then negate.
Step 2: Using $10^k \equiv 10 \pmod{18}$...$(1+1+2+1+3+1+4) \times 10 + 1 = 131 \equiv 5 \pmod{18}$.
Step 3: Convert to positive: $-5 + 18 = 13$.
Answer: **13** ✓

CRISP (1,758 tokens, 56% reduction)

<think>
I need to find $n \equiv -11213141 \pmod{18}$...
$10^k \equiv 10 \pmod{18}$ for $k \geq 1$. So any number mod 18 is: $d_0 + 10 \cdot (d_1 + d_2 + \cdots + d_n)$.
Digits of 11213141: d_7=1, d_6=1, d_5=2, d_4=1, d_3=3, d_2=1, d_1=4, d_0=1.
$d_0 + 10 \cdot (4 + 1 + 3 + 1 + 2 + 1 + 1)$
$= 1 + 10 \cdot 13 = 131$
131 mod 18 = 5 ($18 \times 7 = 126$)
So $-11213141 \equiv -5 \equiv 13 \pmod{18}$. $n = 13$.
CRT verification...mod 2 → 1, mod 9 → 5. Gives $n = 13$. Same result.
</think>

Answer: **13** ✓

Figure 12: **Full model outputs: base Qwen3-8B vs. CRISP** on three MATH-500 problems of increasing difficulty. Each output consists of hidden reasoning (between <think> and </think>) followed by the visible answer (below the gray rule). Blue text highlights redundancy: within reasoning (self-doubt, re-derivation, verification) and in the visible answer (the base model repeats the full derivation as a formatted step-by-step solution, while CRISP outputs only the answer). **Top:** an easy algebra problem (84% reduction). **Bottom:** a number theory problem (56% reduction). Problem 3 continues in Figure 13.

Problem 3 (Algebra/Number Theory): *Let n be a positive integer. Simplify* $\frac{(2^4+\frac{1}{4})(4^4+\frac{1}{4})\cdots[(2n)^4+\frac{1}{4}]}{(1^4+\frac{1}{4})(3^4+\frac{1}{4})\cdots[(2n-1)^4+\frac{1}{4}]}$.

Base model (6,639 tokens)

<think>
...I need to simplify this big fraction...
Let me try to compute a few terms for small n... For n=1: $16.25/1.25 = 13$. Hmm, 13. Let me try n=2. $16.25 \times 256.25/(1.25 \times 81.25) = 13 \times 41/13 = 41$.
Maybe there's a telescoping product? Let me think about how to factor $x^4 + 1/4$... Sophie Germain identity: $a^4 + 4b^4 = (a^2+2ab+2b^2)(a^2-2ab+2b^2)$. Let me check...compute it properly...
With b=1/2: $x^4 + 1/4 = (x^2+x+1/2)(x^2-x+1/2)$.
Numerator factor: $A_k = 4k^2+2k+1/2$, $B_k = 4k^2-2k+1/2$.
Denominator factor: $C_k = 4k^2-2k+1/2 = B_k$. So B_k cancels!
$D_k = 4k^2-6k+5/2 = A_{k-1}$. Telescopes: $\prod A_k / \prod A_{k-1} = A_n/A_0$.
$A_n/A_0 = (4n^2+2n+1/2)/(1/2) = \mathbf{8n^2+4n+1}$.
Let me check: n=1: 8+4+1=13. Correct. n=2: 32+8+1=41. Correct.
Let me see if it factors. Discriminant is $16 - 32 = -16$, so it doesn't factor.
</think>

To simplify the given expression, we begin by analyzing the structure...
Step 1: Use the Sophie Germain Identity: $x^4 + 1/4 = (x^2+x+1/2)(x^2-x+1/2)$.
Step 2: Apply the Factorization to numerator and denominator terms...the term $\prod(4m^2-2m+1/2)$ cancels.
Step 3: Observe the Telescoping Pattern: $D_k = A_{k-1}$...only A_n/A_0 remains.
Step 4: Final Simplification: $(4n^2+2n+1/2)/(1/2) = 8n^2+4n+1$.
Answer: $\mathbf{8n^2+4n+1}$ ✓

CRISP (3,202 tokens, 52% reduction)

<think>
I need to simplify this big fraction... I remember that $a^4 + 4b^4$ can be factored using the Sophie Germain identity.
With $b = 1/2$: $x^4 + 1/4 = (x^2+x+1/2)(x^2-x+1/2)$.
Numerator: $A_k = 4k^2+2k+1/2$, $B_k = 4k^2-2k+1/2$.
Denominator: first factor is $4k^2-2k+1/2 = B_k$, second is $D_k = 4k^2-6k+5/2$.
Since $B_k = C_k$, those cancel: ratio is $\prod A_k / \prod D_k$.
$A_{k-1} = 4(k-1)^2+2(k-1)+1/2 = 4k^2-6k+5/2 = D_k$.
Telescopes: $A_n/A_0 = (4n^2+2n+1/2)/(1/2) = \mathbf{8n^2+4n+1}$.
Check: n=1: 13. n=2: 41. Correct.
</think>

Answer: $\mathbf{8n^2+4n+1}$ ✓

Figure 13: **Full model outputs (continued from Figure 12): a hard product-simplification problem** requiring the Sophie Germain identity and a telescoping argument (52% reduction). The base model arrives at the answer only after extensive numerical exploration, repeated verification, and an attempt to factor the result; after </think>, it reproduces the full four-step derivation. CRISP applies the identity immediately, identifies the telescoping pattern, and outputs only the answer. The harder problem retains more reasoning than the easy (84%) and moderate (56%) cases in Figure 12, illustrating difficulty-adaptive compression (Section 3.3).